

Dokumentation Programm Frequenzanalyse

Juni 2024

Inhaltsverzeichnis

1	Einleitung	2
2	Anwendung des Programms	3
3	Input	4
4	Output - Interpretation der Resultate	11
5	Fehlermeldungen	15
6	Funktionsweise des Programms	18
6.1	Programmstart: Bibliotheken und Umgang mit Excel-Dateien	18
6.2	Benutzereingaben einlesen und kontrollieren	19
6.3	Berechnung der Windlast und Flächenträgheitsmomente	23
6.4	Statische Deformation und Eigenfrequenzen bei starrer Einspannung	27
6.5	Berechnung der Steifigkeit der Einspannung	32
6.6	Statische Deformation und Eigenfrequenzen mit tatsächlicher Ein- spannung	33
6.7	Programmabschluss	34
	Literaturverzeichnis	35
A	Anhang	36

1 Einleitung

Das Python-Programm „Frequenzanalyse“ berechnet die Eigenfrequenzen, die Eigenmodi und die statischen Deformationen von Beleuchtungsmasten.

Es funktioniert jeweils mit einem Input und Output in Form einer Excel-Datei. In den dafür vorgegebenen Zellen können die nötigen Informationen über den zu untersuchenden Beleuchtungsmast eingefügt werden. Das Programm führt anschliessend die Berechnung durch und erstellt automatisch eine neue Excel-Datei, in welcher die Resultate dargestellt werden. Ebenfalls wird der Input auf seine Plausibilität überprüft und mögliche Fehler werden in der Output-Datei aufgeführt. Das Programm führt eine Mastmodellierung unter der Annahme einer biegesteifen Einspannung im Boden durch. Dabei werden die statischen Deformationen für den Grenzzustand der Gebrauchstauglichkeit, sowie die Eigenfrequenzen und Eigenmodi in zwei zueinander orthogonale Richtungen bestimmt. Anschliessend wird durch die vom Benutzer eingegebene, gemessene erste Eigenfrequenz der tatsächliche Wert der Steifigkeit der Einspannung im Boden berechnet. Mithilfe dieser zusätzlichen Information wird der Mast erneut modelliert um die genaueren Werte der statischen Deformation, der Eigenfrequenzen und Eigenmodi zu erhalten.

Die neu berechneten Eigenfrequenzen höherer Ordnung können mit den gemessenen Frequenzen verglichen werden, um Rückschlüsse auf mögliche Schäden des Mastes zu erhalten. Zusätzlich könnten Schäden im Gründungsbereich einen Einfluss auf die Steifigkeit der Einspannung haben.

2 Anwendung des Programms

Zur Anwendung des Programms werden zwei Dateien benötigt: Die Excel-Datei für den Input mit dem Namen „Dateninput“ und das Python-Programm „Frequenzanalyse“ selbst. Zusätzlich muss die Programmiersprache Python auf dem Gerät installiert sein. Auf die beiden Dateien kann über die im Anhang A aufgeführte Polybox zugegriffen werden. Die Input-Datei wird im Kapitel 3 erklärt. Bereits hier soll erwähnt werden, dass die Berechnungsdauer des Programms stark abhängig ist von den Benutzereingaben und der zur Verfügung stehenden Rechnerleistung. Die Input-Datei muss im gleichen Ordner wie das Python-Programm gespeichert werden und ihr Name sollte nicht verändert werden. Ansonsten ist eine leichte Anpassung am Anfang des Codes erforderlich. Dies ist möglich, wird aber nicht empfohlen. Vor der Ausführung des Programms muss die Input-Datei gespeichert werden, da das Python-Programm auf die gespeicherte Version und nicht auf eine aktuell geöffnete Version zugreifen kann. Es befinden sich zwei Versionen des Programms „Frequenzanalyse“ in der Polybox: eine *.ipybn*- und eine *.py*-Datei. Die *.ipybn*-Datei wurde zur Entwicklung verwendet. Benutzer mit Programmiererfahrung können die *.ipybn*-Datei auf Jupyter Notebook verwenden oder die *.py*-Datei mit einem gebräuchlichen Editor. Für Benutzer ohne Programmiererfahrung wird empfohlen die Eingabeaufforderung zu verwenden. Dazu gibt man „cmd“ in die Suchzeile ein. Dann öffnet man die Eingabeaufforderung. Anschliessend gibt man „cd“ in die Eingabeaufforderung ein, gefolgt von einem Leerschlag und dann dem Dateipfad zum Speicherort der Python-Datei. Dabei muss die Datei selbst nicht angegeben werden. Das letzte Element des Dateipfads sollte also der Ordner sein, in welchem sich die Datei befindet. Dann drückt man die Entertaste. Anschliessend gibt es noch eine zweite Eingabe: „python“ - Leertaste - „Frequenzanalyse.py“ - Entertaste. Danach wird das Programm ausgeführt und die Output-Datei erscheint unter dem Namen „Resultate.Frequenzanalyse“ im gleichen Ordner wie die Input-Datei und das Python-Programm. Möglicherweise muss der Ordner einmal aktualisiert werden. Der Name und Speicherort der Output-Datei einer alten Berechnung sollten geändert werden, bevor man eine neue durchführt. Ansonsten werden die alten Resultate durch die der neuen Berechnung ersetzt und man kann nicht mehr auf sie zugreifen.

3 Input

Die Anwendung der Python-Programme „Frequenzanalyse“ und „Standicherheit“ erfolgt für beide mithilfe der in den folgenden Abbildungen gezeigten Excel-Datei „Dateninput“. Die Input-Datei sollte bereits aufgrund der Beschreibungen in den jeweiligen Zellen bedienbar sein. Insbesondere die Einheit der Eingabe wird immer in der darüber oder links daneben liegenden Zelle vorgeschrieben. Hier folgt nun eine detaillierte Anleitung. Wichtig ist, den Input immer in die farbig markierten Zellen zu schreiben. Alle anderen Zellen sollten nicht verändert werden und haben keinen Einfluss auf die Berechnungen. Zusätzlich dürfen die Zellen nicht verschoben werden.

	A	B	C	D	E	F
1						
2	1. Form	Form				
3		konisch (Achteck)				
4	2. Material	Material				
5		Stahl S235				
6						
7	3. Abmessungen	Masthöhe h [m]				
8		15.5				
9		D_1 [m]	D_2 [m]	D_3 [m]		...
10		0.115	0.034			
11		in z_1 [m] =	in z_2 [m] =	in z_3 [m] =	...	...
12	von unten nach oben:	0	15.5			
13		Dicke t [m]				
14		0.004				
15						
16	4. Leuchte (am oberen Mastende)	Fläche Leuchte, x-Richtung [m^2]	Fläche Leuchte, y-Richt	Masse Leuchte [kg]	Ausmitte x [m]	Ausmitte y [m]
17		0.8	0.25	80	0.5	0.5

Abbildung 3.1: Input-Datei, Zeilen 1-17: Form, Material und allgemeine Geometrie von Mast und Leuchte

Die Abbildung 3.1 zeigt den ersten Abschnitt der Input-Datei.

Zelle B3: Hier kann die Form des Mastes festgelegt werden. Es existieren drei Auswahlmöglichkeiten: „konisch (Hohlkreis)“, „konisch (Achteck)“, „zylindrisch (Hohlkreis)“. Der erste Begriff bezieht sich auf die Art der Verjüngung des Mastes. Nimmt der Durchmesser kontinuierlich und gleichmässig ab, so ist „konisch“ auszuwählen, ist der dagegen nach oben abgestuft mit einzelnen Abschnitten mit jeweils konstantem Durchmesser, so ist „zylindrisch“ die richtige Wahl. Falls sich der Durchmesser des Mastes nicht ändert, sind beide Optionen in Ordnung. Der Begriff in Klammern steht für die Form des Querschnitts. Diese ist bei Betrachtung des Mastes direkt erkennbar.

Zelle B5: Für das Mastmaterial stehen „Stahl S235“, „Stahl S275“, „Stahl S355“ und „Aluminium 6061“ zur Verfügung. Die Stahlgüte kann durch Herstellerangaben oder durch eine Festigkeitsprüfung bestimmt werden.

Zelle B8: In dieser Zelle sollte die Höhe des Mastes eingegeben werden. Da die Norm SN EN 40-3-3 [1], auf welchen die Berechnung der Windlast beruht, ausschliesslich auf Masten mit einer Höhe bis 20 m anwendbar ist, ist bei höheren Masten nur ein Teil der Resultate gültig. Darauf wird im Kapitel 4 hingewiesen.

Zeilen 10 und 12: Hier können im Falle eines runden Querschnitts die Durchmesser und im Falle eines achteckigen Querschnitts die Seitenlängen des Achtecks festgelegt werden. In der Zeile 12 sollte immer die zum jeweiligen Querschnitt bzw. Seitenlänge zugehörige Höhe über dem Boden eingegeben werden. Dabei ist darauf zu achten, dass man links mit der tiefsten Höhe (und damit dem grössten Durchmesser bzw. Seitenlänge) beginnt. Bei konischen Masten müssen nur die Spalten B und C ausgefüllt werden. Bei zylindrischen Masten müssen mindestens die Spalten B und C ausgefüllt werden, es können aber unbegrenzt weitere Durchmesser eingeführt werden. Hier ist der Input nicht auf die farbig markierten Zellen begrenzt. Besonders wichtig ist es bei zylindrischen Masten, dass die einzugebende Höhe immer der Höhe der Unterkante des entsprechenden Mastabschnitts entspricht. Dies ist in Abbildung 3.2 dargestellt.

Falls der Durchmesser über den gesamten Mast konstant bleibt, sollte der Durchmesser bzw. die Seitenlänge in die Zellen B10 und C10 eingefügt werden. Die Zellen B12 und C12 sollten ebenfalls ausgefüllt werden, die konkreten Werte sind dabei aber irrelevant.

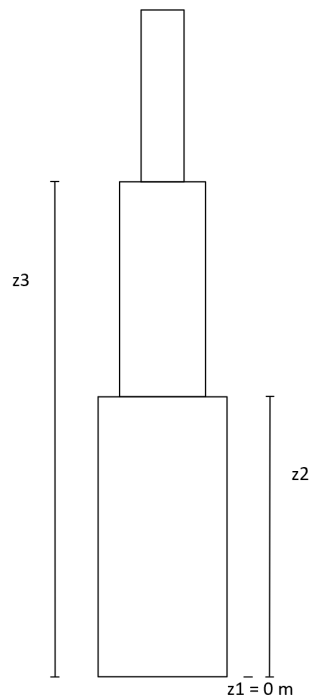


Abbildung 3.2: Darstellung der unterschiedlichen Höhen

Zelle B14: Hier kann die Dicke der Mastwand eingegeben werden. Es wird davon ausgegangen, dass diese über den gesamten Mast konstant bleibt. Es ist besonders auf die Einheit (Meter) zu achten.

Zellen B17-D17: Hier können Informationen über die Leuchte an der Mastspitze eingegeben werden. Zum ersten Mal müssen die zueinander orthogonalen x- und y-Richtung festgelegt werden. Falls der Mast eine Öffnung (Tür) aufweist, sollte darauf geachtet werden, dass man eine der Richtungen so definiert, dass die Öffnung in Achsenrichtung orientiert ist. Ein Beispiel ist in der Abbildung 3.3 gegeben. Es ist dabei aber noch nicht relevant, ob die Öffnung in x- oder y-Richtung zeigt. Bei allen folgenden Angaben in der Input-Datei, bei welchen die Richtung eine Rolle spielt, kann man sich vereinfacht vorstellen, dass man aus dieser Richtung auf den

Mast schaut. (Die Ausnahme sind die gemessenen Eigenfrequenzen, welche separat erklärt werden.) Die „Fläche der Leuchte, x-Richtung“ in Zelle B17 bedeutet dementsprechend, dass man den Mast aus Richtung der x-Achse betrachtet und die dabei sichtbare (auf die yz-Ebene projizierte) Fläche eingibt. Die Zelle C17 funktioniert analog. In der Zelle D17 folgt die Masse der Leuchte.

Zellen E17 und F17: Die „Ausmitte“ der Leuchte muss nicht eingegeben werden. Diese Information bezieht sich auf den Abstand der Leuchte zur Mastachse und ist nur für das Programm „Standicherheit“ relevant, um die Torsionsmomente zu bestimmen.

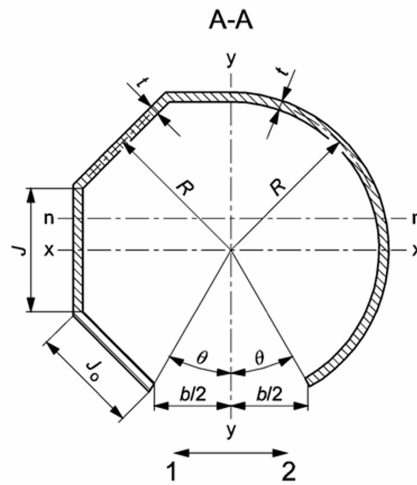


Abbildung 3.3: Orientierung der Öffnung [1]

	A	B	C	D	E	F	G	H
18								
19								
20	5. Zusatzmassen		Einzelmasse 1	Einzelmasse 2	Einzelmasse 3	Einzelmasse 4	...	...
21		Höhe z über GOK [m]	1.2	4.3	10			
22		Projizierte Fläche A _p , x-Richtung [m²]	0.25	0.03	0.1			
23		Projizierte Fläche A _p , y-Richtung [m²]	0	0	0.25			
24		Masse [kg]	22	12	10			
25		Ausmitte x [m]	0	0	0.5			
26		Ausmitte y [m]	0	0	-0.25			
27								
28	6. Türen		Tür 1	Tür 2	Tür 3	Tür 4	...	...
29		Verstärkung	Ja (verstärkt Typ 4)	Ja (unverstärkt)	Nein	Nein	Nein	Nein
30		Höhe z unten [m]	0.46	2				
31		Gesamtlänge a [m]	0.6	0.5				
32		Breite b [m]	0.14	0.08				
33		Dicke der Verstärkung t _w [m]	0.015					
34		Abmessung Verstärkung d _w [m]	0.03					
35		Eckradius N [m] (falls bekannt)	0.00045					
36		Ausrichtung	x	y	x	x	x	x

Abbildung 3.4: Input-Datei, Zeilen 18-36: Geometrie von Zusatzmassen und Türen

Die Abbildung 3.4 zeigt den mittleren Teil der Input-Datei.

Zeilen 21-24: Hier können die Zusatzmassen definiert werden. In der Spalte C sollte die erste Zusatzmasse stehen, in der Spalte D die zweite etc. Es gibt keine Beschränkung der Anzahl Zusatzmassen. Die „Höhe z über GOK“ in der Zeile 21 bezeichnet die Höhe des Mittelpunkts beziehungsweise Schwerpunkts der Zusatzmasse über dem Boden. In den Zeilen 22-24 können die projizierten Flächen und die Masse eingefügt werden. Das Vorgehen ist dabei dasselbe wie in der Zeile 17.

Zeilen 25 und 26: Diese Zeilen müssen nicht ausgefüllt werden. Hier kann bei Verwendung des Programms „Standicherheit“ die „Ausmitte“ angegeben werden (horizontaler Abstand der Mittelpunkte der Zusatzmassen von der Mastachse).

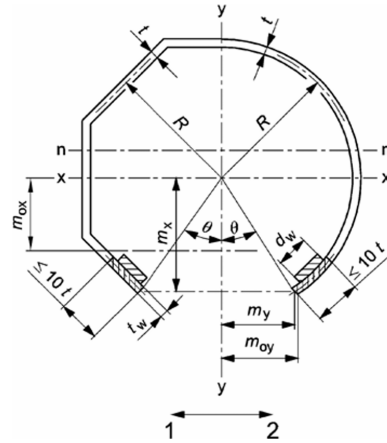


Abbildung 3.5: Öffnung verstärkt, Typ 4 [1]

Zeilen 29-36: Hier können die Türen definiert werden. Die unterste Tür sollte in die Spalte C eingegeben werden, die nächste in die Spalte D etc. Die Anzahl Türen ist nicht beschränkt, allerdings dürfen sich nie zwei Türen auf der gleichen Höhe befinden, da die Norm SN EN 40-3-3 [1] in diesen Fällen keine Berechnung der Biege- und Torsionswiderstandsmomente erlaubt. In der Zeile 29 kann für jede Tür ausgewählt werden, ob sie unverstärkt ist oder eine Verstärkung des Typs 4 gemäß SN EN 40-3-3, Bild 6d [1] (siehe Abbildung 3.5 aufweist. Falls keine weiteren Türen mehr eingegeben werden sollen, muss „Nein“ ausgewählt werden. In der Zeile 30 sollte die Höhe der Unterkante der Tür eingefügt werden. Die „Gesamtlänge“ in der Zeile 31 bezieht sich auf die Höhe der Tür. Die Breite der Tür bezeichnet selbsterklärend die horizontale Abmessung. Die Zeilen 33 und 34 (Dimensionen der Verstärkung müssen nur ausgefüllt werden, falls „Ja (verstärkt Typ 4)“ gewählt wurde. Der Eckradius der Tür (siehe Abbildung 3.6) kann in der Zeile 35 angegeben werden. Falls er nicht bekannt ist, kann die Zelle auch leer gelassen oder es kann konservativ 0 eingesetzt werden.

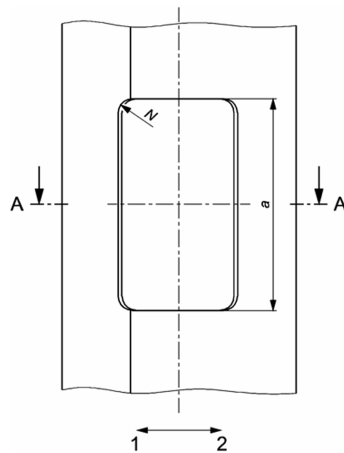


Abbildung 3.6: Eckradius der Tür: N [1]

	A	B	C
37			
38	7. Windzone	Windzone (CH)	
39		1 : q _{p0} = 0.9 kN/m ² , 'Allgemein'	
40			
41	8. Geländekategorie	Geländekategorie gemäss SIA 261, 6.2.1.2, Tabelle 4	
42		2a = grosse Ebene	
43			
44	9. Lastbeiwerte (für Nachweis der Tragsicherheit)		
45		SIA 260	
46			
47	10. Klasse hor. Verformung	Klasse gemäss SN-E 40-3-3, Tabelle 4	
48		Klasse 2 (6%)	
49			
50	11. Elemente für Berechnung	Anzahl Elemente	
51		100	
52			
53	12. gemessene Eigenfrequenzen	1. Eigenfrequenz x [Hz]	1. Eigenfrequenz y [Hz]
54	Falls unbekannt, bitte 0 eingeben	0.73	0.77

Abbildung 3.7: Input-Datei, Zeilen 37-54: Standort- und normative Beiwerte

Die Abbildung 3.7 zeigt den letzten Abschnitt der Input-Datei.
Zelle B39: Hier kann man die Windzone beziehungsweise den Referenzwert des Staudrucks auswählen. Dieser kann in der SIA 261 (2014) im Anhang E [2] auf der Karte abgelesen werden (siehe Abbildung 3.8).

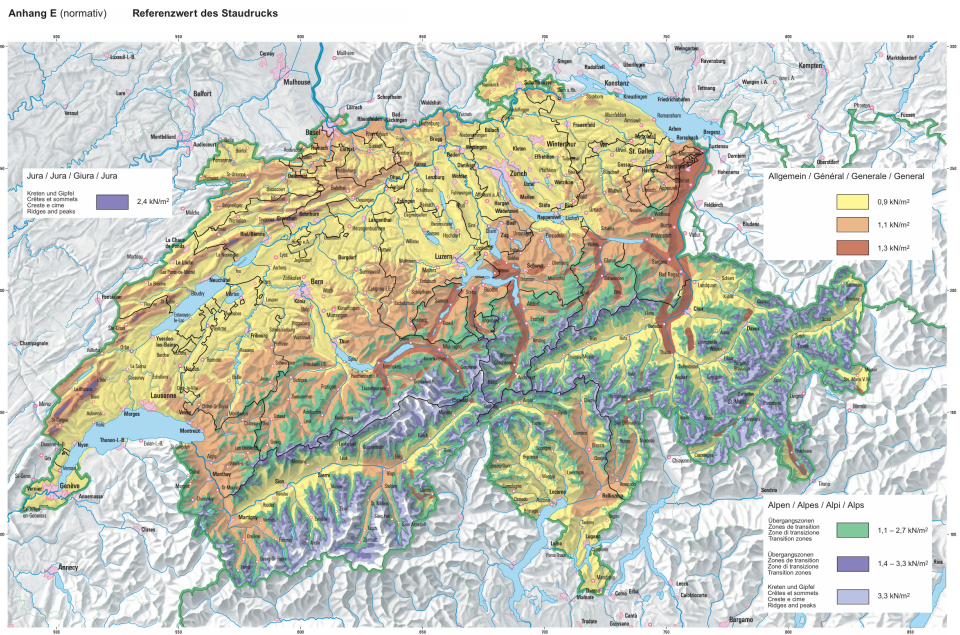


Abbildung 3.8: Referenzwert des Staudrucks gemäss SIA 261 [2]

Zelle B42: Hier kann die Geländekategorie gemäss SIA 261, 6.2.1.2 [2] ausgewählt werden (siehe Tabelle 3.1).

Tabelle 3.1: Geländekategorien mit Beispielen gemäss SIA 261[2]

Geländekategorie	Beispiele
II	Seeufer
IIa	grosse Ebene
III	Ortschaften, freies Feld
IV	grossflächige Stadtgebiete

Zelle B45: Hier könnte bestimmt werden, gemäss welcher Norm die Lastbeiwerte für den Nachweis der Tragsicherheit gewählt werden. Allerdings ist diese Information für die Berechnung nicht relevant. Sie wird nur für das Programm „Standicherheit“ verwendet.

Zelle B48: In dieser Zelle kann festgelegt werden, wie viel Prozent der Masthöhe die Deformation der Mastspitze maximal betragen darf. Die zur Verfügung stehenden Klassen werden in der SN EN 40-3-3, in Tabelle 4 [1] definiert (siehe Abbildung 3.9).

Tabelle 4 — Grösste horizontale Verformung

Klasse	Grösste horizontale Verformung
1	0,04 ($h + w$)
2	0,06 ($h + w$)
3	0,10 ($h + w$)

Abbildung 3.9: Klassen der erlaubten horizontalen Verformungen gemäss SN EN 40-3-3 [1]

Zelle B51: Hier kann gewählt werden in wie viele diskrete Elemente der Mast während der Berechnung unterteilt werden soll. Eine grössere Anzahl Elemente erhöht die Genauigkeit der Berechnung, aber auch die Berechnungsdauer. Deshalb ist die Anzahl Elemente auf 1'000 beschränkt. Ein höherer Wert darf nicht eingegeben werden. Die Abhängigkeit der Berechnungsdauer von der Anzahl Elemente bei Verwendung eines „HP Spectre“-Laptops (Einfluss der Rechenleistung) soll im Folgenden verdeutlicht werden: Mit 100 Elementen ist von einer Berechnungsdauer von ca. 20 Sekunden auszugehen, bei 1'000 Elementen dagegen kann die Berechnung über 30 Minuten dauern. Da ein Teil des Programms eine iterative Berechnung enthält, lässt sich die Berechnungsdauer aber nicht absolut in Abhängigkeit der Elemente bestimmen. Auch bei gleicher Anzahl an Elementen, aber anderen Unterschieden im Input können die Berechnungsdauern deutlich variieren. Da die Masthöhe auf 20 m beschränkt ist, sind die Unterschiede der Resultate bei 100 und bei 1'000 Elementen marginal. Dementsprechend ist eine sehr grosse Anzahl Elemente nur selten zielführend.

Zellen B54 und C54: In diesen Zellen können die gemessenen ersten Eigenfrequenzen in x- und y-Richtung eingegeben werden. Die Eigenfrequenz in x-Richtung bezieht sich dabei auf eine Auslenkung des Mastes in x-Richtung und nicht auf eine Schwingung, deren Auslenkung aus der x-Richtung beobachtet werden kann. Falls beide Zellen mit Werten ungleich 0 ausgefüllt werden, wird in der Berechnung die Eigenfrequenz in x-Richtung verwendet, um die Steifigkeit der Einspannung zu ermitteln. Falls man die Eigenfrequenz nur in die y-Richtung kennt, kann man in die Zelle B54 0 einsetzen und die bekannte Eigenfrequenz in die Zelle C54. Dann wird die Eigenfrequenz in y-Richtung für die Berechnung verwendet. Falls keine ersten Eigenfrequenzen bekannt sind, sollte 0 in beide Zellen eingesetzt werden. In diesem Fall wird nur der Teil des Programms ausgeführt, welcher von einer absolut biegesteifen Einspannung ausgeht. Eine Bestimmung der Steifigkeit der Einspannung ist

dann nicht möglich.

4 Output - Interpretation der Resultate

Ebenso wie der Input erfolgt auch der Output in Form einer Excel-Datei. Wenn die Berechnung abgeschlossen ist, werden die Resultate automatisch in eine Excel-Datei mit dem Namen „Resultate_Frequenzanalyse“ geschrieben. Diese Output-Datei wird im gleichem Ordner wie das Python-Programm gespeichert. Wie auch beim Input ist es möglich, den Namen und Speicherort der Datei automatisch zu verändern. Dazu müsste die letzte Zeile des Codes angepasst werden. Es wird aber empfohlen, dies nicht zu tun.

	A	B	C	D
1				
2		Ergebnisse Mastmodellierung		
3				
4	zulässige horizontale Verschiebung [mm]:	930		
5	horizontale Verschiebung, x-Richtung [mm]:	891.85		
6	horizontale Verformungen i.O.			
7	Eigenfrequenzen in x-Richtung:			
8	Eigenfrequenz	1 0.7668		Hz
9	Eigenfrequenz	2 3.8886		Hz
10	Eigenfrequenz	3 10.5314		Hz
11	Eigenfrequenz	4 20.8872		Hz
12	Eigenfrequenz	5 35.5682		Hz
13	zulässige horizontale Verschiebung [mm]:	930		
14	horizontale Verschiebung, y-Richtung [mm]:	564.67		
15	horizontale Verformungen i.O.			
16	Eigenfrequenzen in y-Richtung:			
17	Eigenfrequenz	1 0.7693		Hz
18	Eigenfrequenz	2 3.9069		Hz
19	Eigenfrequenz	3 10.5731		Hz
20	Eigenfrequenz	4 20.9428		Hz
21	Eigenfrequenz	5 35.6264		Hz

Abbildung 4.1: Output-Datei, Zeilen 1-21

Die Abbildung 4.1 zeigt den ersten Teil der Resultate. Dieser Abschnitt wurde mit der Annahme einer perfekt biegesteifen Einspannung berechnet.

In der Zelle B4 wird die maximal erlaubte horizontale Verschiebung der Mastspitze aufgeführt. Je nachdem, welche Klasse gemäss SN EN 40-3-3, Tabelle 4 [1] der Benutzer beim Ausfüllen des Input-Dokuments angegeben hat, beträgt diese 4%, 6% oder 10% der Masthöhe. In den Zeilen darunter folgen Resultate der Berechnung in x-Richtung und anschliessend in y-Richtung. Zelle B5 zeigt die statische Verschiebung der Mastspitze in x-Richtung bei Windlasten (ebenfalls in x-Richtung) auf charakteristischem Niveau. Dieses Resultat gilt nur für Masten mit einer Höhe bis 20 m, da bei der Berechnung die Norm SN EN 40-3-1 [3] zur Anwendung kommt. Diese Verschiebung wird auch im Programm „Standicherheit“ berechnet. „Frequenzanalyse“ verwendet dazu die direkte Steifigkeitsmethode (Matrizenrechnung),

während „Standicherheit“ die Kraftmethode verwendet. Die Differenzen der Resultate bewegen sich für die geprüften Masten im Bereich von Zehntelmillimetern. Relativ zur ermittelten Verschiebung lagen die Abweichungen dementsprechend bei unter 0.05%. Falls ein Benutzer also nur an der statischen Deformation interessiert ist, ist es unerheblich welches Programm verwendet wird. In der Zelle A6 wird schlussendlich angezeigt, ob die Anforderung an den Grenzzustand der Gebrauchstauglichkeit in x-Richtung erfüllt ist.

Anschliessend folgen in den Zellen C8 bis C12 die erste bis fünfte Eigenfrequenz des Mastes. Dabei ist zu beachten, dass die Berechnung tieferer Frequenz mit einer grösseren Genauigkeit möglich ist als die der höheren. zusätzlich kann man Normalfall kann man beobachten, dass die in der Zelle C8 aufgeführte erste Eigenfrequenz nicht der tatsächlich gemessenen Eigenfrequenz entspricht. Dies ist der Fall, weil ein realer Mast nicht perfekt biegesteif eingebaut ist. Dadurch sind die gemessenen Frequenzen jeweils tiefer als die Resultate dieses Abschnitts. Ab der Zeile 12 folgen die gleichen Resultate bei Betrachtung des Mastes in y-Richtung.

	A	B	C	D
22				
23	Resultate unter Berücksichtigung der Einspannung im Boden:			
24	Federsteifigkeit der Einspannung [kNm/rad]:	7510		
25	zulässige horizontale Verschiebung [mm]:	930		
26	horizontale Verschiebung, x-Richtung [mm]:	998.24		
27	horizontale Verformungen zu gross.			
28	Eigenfrequenzen in x-Richtung:			
29	Eigenfrequenz	1	0.7296	Hz
30	Eigenfrequenz	2	3.5947	Hz
31	Eigenfrequenz	3	9.7548	Hz
32	Eigenfrequenz	4	19.5802	Hz
33	Eigenfrequenz	5	33.5669	Hz
34	zulässige horizontale Verschiebung [mm]:	930		
35	horizontale Verschiebung, y-Richtung [mm]:	641.89		
36	horizontale Verformungen i.O.			
37	Eigenfrequenzen in y-Richtung:			
38	Eigenfrequenz	1	0.7318	Hz
39	Eigenfrequenz	2	3.6067	Hz
40	Eigenfrequenz	3	9.7724	Hz
41	Eigenfrequenz	4	19.5926	Hz
42	Eigenfrequenz	5	33.572	Hz

Abbildung 4.2: Output-Datei, Zeilen 22-42

Der zweite Teil der Resultate ist in der Abbildung 4.2 aufgeführt. Dieser Abschnitt des Outputs entsteht nur wenn beim Input in den Zellen B54 und C54 mindestens eine gemessene erste Eigenfrequenz eingegeben wurde. Falls der Benutzer dagegen in beide Zellen 0 eingegeben hat, kann dieser Teil der Berechnungen nicht durchgeführt werden. Für diese Resultate wurde die Steifigkeit der Einspannung so bestimmt, dass die erste Eigenfrequenz der Modellierung des Programms mit der gemessenen ersten Eigenfrequenz übereinstimmt bzw. eine Abweichung von unter 0.001 Hz aufweist. Wenn möglich, wird dabei die Eigenfrequenz in x-Richtung verwendet. Falls diese aber unbekannt ist (0 in Input-Datei Zelle B54), dann wird die y-Richtung betrachtet.

In der Zelle B24 ist die berechnete Steifigkeit der Einspannung im Boden aufgeführt. Grosse Werte entsprechen einer sehr steifen Einspannung, die nur wenig Rotation im Fusspunkt des Mastes erlaubt. Das Programm berechnet zwar den Wert der Steifigkeit, eine Einordnung des Resultats muss der Benutzer aber selbst vornehmen. Abhängig von der Geometrie des Mastes, der konstruktiven Ausbildung des Fuss-

punkts und dem verwendeten Material kann die Steifigkeit stark variieren. Sehr tiefe Werte für die Steifigkeit deuten dabei auf mögliche Schäden im Gründungsbereich.

In der Zelle B26 wird die statische Deformation der Mastspitze in x-Richtung aufgeführt. Ein Vergleich mit dem ersten Resultat in Zelle B5 (siehe Abbildung 4.1) zeigt, dass sich die Verschiebung durch die genauere Mastmodellierung vergrößert hat. Auch in diesem Abschnitt wird wieder die Anforderung an den Grenzzustand der Gebrauchstauglichkeit bewertet. In der Zelle A28 ist in diesem Fall aufgeführt, dass die horizontale Verformung zu gross ist, weil der Grenzwert (Zelle B25) überschritten wurde. In den Zeilen 34 bis 36 befinden sich die entsprechenden Resultate in y-Richtung. Erneut gelten die Resultate der statischen Deformation nur für Masten bis 20 m.

In den Zeilen 29 bis 33 sind die ersten fünf Eigenfrequenzen in x-Richtung aufgeführt. Die erste Eigenfrequenz entspricht dabei der gemessenen. (Ausser es wurde keine gemessene Eigenfrequenz eingegeben.) Mithilfe der anderen Eigenfrequenzen, insbesondere der zweiten, können dagegen Rückschlüsse auf den Zustand des Mastes gemacht werden. Wenn die berechnete zweite oder dritte Eigenfrequenz von der gemessenen abweicht, deutet dies auf mögliche Schäden am Mastchaft hin. Dasselbe gilt auch für höhere Eigenfrequenzen. Allerdings ist bei Verwendung der Mastap-Software normalerweise nur maximal die dritte Eigenfrequenz erkennbar und die Genauigkeit der berechneten Werte sinkt für höhere Frequenzen. Falls ein Benutzer an den Berechnungsergebnissen von noch höheren Eigenfrequenzen als der fünften interessiert ist, liesse sich das vergleichsweise simpel implementieren, da das Programm diese ohnehin bereits berechnet. Nur auf die Darstellung dieser Frequenzen wurde verzichtet.

In den Zeilen 38 bis 42 sind die berechneten Eigenfrequenzen in y-Richtung abgebildet. Falls die Berechnung der Steifigkeit der Einspannung mittels Eigenfrequenz in x-Richtung stattgefunden hat, dann kann auch die y-Eigenfrequenz Abweichungen vom Messresultat aufweisen. Das kann beispielsweise geschehen, wenn die Eigenschaften der Einspannung im Boden (aufgrund von Schäden wie Korrosion etc.) nicht in alle Richtungen konstant sind. Zusätzlich können auch wieder die höheren Eigenfrequenzen betrachtet werden. Insbesondere falls die erste Eigenfrequenz auch in y-Richtung dem Messresultat nahe ist, können wiederum Schäden im Mast oberhalb des Gründungsbereichs detektiert werden. Auch bei dieser Analyse der Eigenfrequenzen muss beachtet werden, dass die Messresultate zwar Hinweise geben können, die möglichen Ursachen der Abweichungen der Eigenfrequenzen wurden aber während der Entwicklung des Programms nicht genügend genau untersucht, um differenzierte Aussagen zu erlauben. Dementsprechend kann das Programm die Relevanz möglicher Schäden nicht angeben. Dafür ist Fachwissen des Benutzers ebenso gefordert wie die Berücksichtigung von Material und Geometrie des Mastes und der Einspannung.

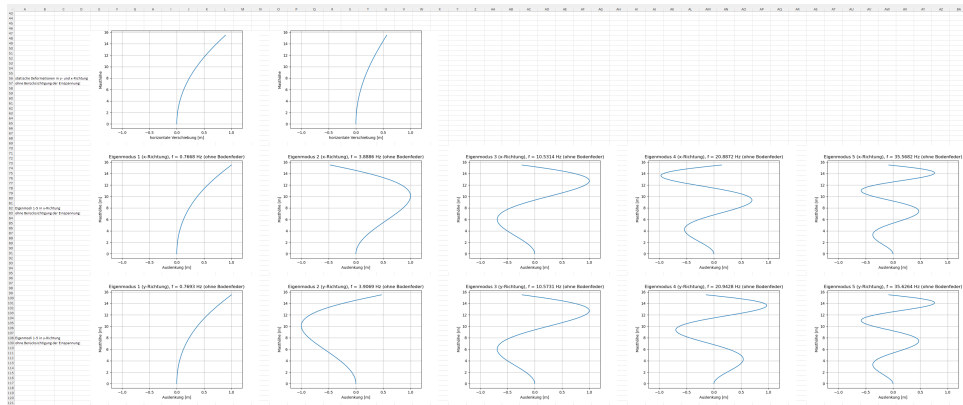


Abbildung 4.3: Output-Datei, Zeilen 43-121

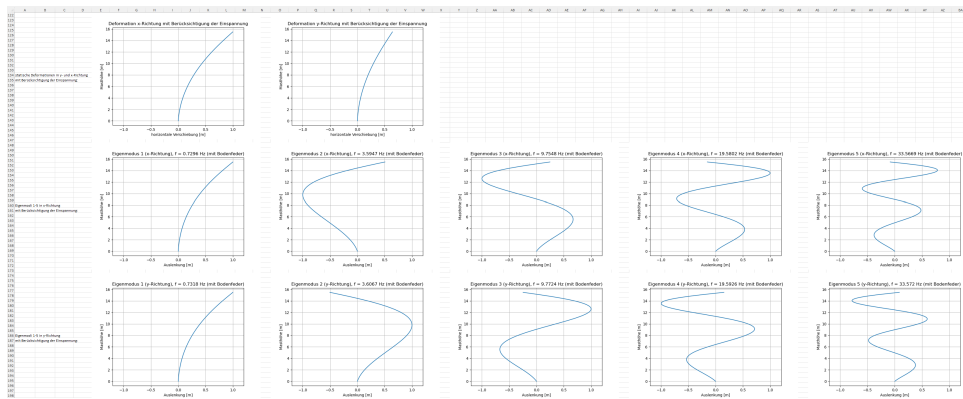


Abbildung 4.4: Output-Datei, Zeilen 122-198

Die Abbildungen 4.3 und 4.4 zeigen den dritten und letzten Abschnitt der Resultate. Abbildung 4.3 ist die Visualisierung der Resultate, bei welchen die Steifigkeit der Einspannung nicht berücksichtigt wurde. Zuerst wird die statische Deformation in x- und y-Richtung aufgezeigt und anschliessend die ersten fünf Eigenmodi in x- und danach in y-Richtung. Abbildung 4.4 zeigt das Gleiche unter Berücksichtigung der Einspannung. Auf der linken Seite, hier aufgrund der Grössenverhältnisse nur schwer lesbar, sind die folgenden Beschreibung von oben nach unten aufgeführt:

- „statische Deformation in x- und y-Richtung ohne Berücksichtigung der Einspannung:“
- „Eigenmodi 1-5 in x-Richtung ohne Berücksichtigung der Einspannung:“
- „Eigenmodi 1-5 in y-Richtung ohne Berücksichtigung der Einspannung:“
- „statische Deformation in x- und y-Richtung mit Berücksichtigung der Einspannung:“
- „Eigenmodi 1-5 in x-Richtung mit Berücksichtigung der Einspannung:“
- „Eigenmodi 1-5 in y-Richtung mit Berücksichtigung der Einspannung:“

Dieser Abschnitt dient dazu, die Resultate für den Benutzer verständlicher zu machen. Zudem können insbesondere die statischen Deformationen aufgrund des gleichen Massstabs einfach miteinander verglichen werden.

5 Fehlermeldungen

Bei der Eingabe des Inputs können Fehler begangen werden. Zudem ist das Programm nicht in der Lage jeden erdenklichen Input zu verarbeiten. Um typische Fehler (Bsp. Verwechslung von Einheiten) direkt zu erkennen, wird der Input gleich nach dem Einlesen kontrolliert. Sollte dabei ein Fehler bzw. unrealistische Werte erkannt werden, so führt das Programm die Berechnung nicht aus und schreibt stattdessen eine Fehlermeldung in die Excel-Datei „Resultate_Standsicherheit“. In dieser wird bereits kurz beschrieben, weshalb der Input als fehlerhaft bewertet wurde. In der folgenden Auflistung werden die möglichen Fehlermeldung erklärt.

Zuerst ist es aber wichtig anzumerken, dass bei der Inputkontrolle nicht jeder Fehler erkannt werden kann. Falls der Input einen nicht direkt erkannten Fehler aufweist, führt dies normalerweise dazu, dass das Programm nicht vollständig ausgeführt wird. Es wird dann keine neue Output-Datei erstellt. In einem solchen Fall sollte der Benutzer als erste Massnahme die gesamte Input-Datei kontrollieren.

- „Der Durchmesser bzw. die Seitenlänge wurde für eine zu grosse Höhe eingegeben.“
In der Zeile 12 kann die Höhe ab Boden der verschiedenen Mastdurchmesser bzw. Seitenlängen eingegeben werden. Ein Wert, welcher grösser ist als die Masthöhe führt zu dieser Fehlermeldung, da auf einer solchen Höhe kein Durchmesser existiert, der gemessen werden könnte.
- „Der Durchmesser bzw. die Seitenlänge liegt nicht im Bereich 1-80 cm.“
Um der Verwechslung von Einheiten vorzubeugen, wird nur der erwähnte Bereich zugelassen. Aufgrund von Erfahrungswerten wird davon ausgegangen, dass alle zu untersuchenden Masten in diesem Bereich liegen.
- „Der Durchmesser des Mastes nimmt nach oben zu anstatt ab.“
Diese Fehlermeldung bedeutet, dass in den Zeilen 10 und 12 für eine grössere Höhe auch ein grösserer Durchmesser bzw. Seitenlänge definiert wurde. Es wird aber damit gerechnet, dass sich die Masten nach oben verjüngen oder einen konstanten Querschnitt beibehalten.
- „Die Wandstärke liegt unter 1 mm.“
Die Dicke der Mastwand muss mindestens 1 mm betragen, ansonsten wird die Annahme getroffen, dass es sich beim Input der Zelle B14 um einen Fehler handelt.
- „Die Wandstärke liegt über 20 mm. Bitte überprüfen Sie, dass kein Fehler vorliegt.“
Diese Meldung hat im Gegensatz zu den anderen Fehlermeldungen keinen Abbruch des Programms zur Folge. Die Berechnung wird dennoch durchgeführt.

Zusätzlich zu den Resultaten wird diese Meldung ausgegeben um den Benutzer auf die grosse Wandstärke, welche nur selten vorkommt, aufmerksam zu machen.

- „Bitte beschränken Sie die Anzahl Elemente auf maximal 1'000.“
Die Anzahl Elemente in der Zelle B51 wird auf 1000 beschränkt. Es darf angenommen werden, dass die Genauigkeit der Berechnung bei 1000 Elementen hoch genug ist, da dies einer Elementhöhe von maximal 2 cm entspricht. Mit dieser Beschränkung werden der Berechnungsaufwand und die Berechnungsdauer kontrolliert.
- „Die Länge einer Tür liegt nicht im Bereich 0-1 m.“
Diese Einschränkung entspricht Erfahrungswerten und verhindert zu grosse Türhöhen, die durch die inkorrekte Verwendung von Einheiten beim Ausfüllen der Zeile 30 entstehen könnte.
- „Der Eckradius einer Tür liegt nicht im Bereich 0 bis halbe Türbreite.“
Der Eckradius (siehe Abbildung 3.6) kann selbst im extremsten Fall (obere und untere Enden der Türe als Halbkreis gestaltet) nicht über der halben Türbreite liegen.
- „Die Dicke der Verstärkung einer Tür liegt nicht im Bereich 1-30 mm.“
Auch diese Einschränkung basiert auf Erfahrungswerten und soll den korrekten Gebrauch der Längeneinheiten sicherstellen.
- „Die Breite der Verstärkung einer Tür liegt nicht im Bereich 1-100 mm.“
Für diese Fehlermeldung der Verstärkungsbreite gilt dieselbe Begründung wie bei der Verstärkungsdicke.
- „Die Fläche der Leuchte liegt nicht im Bereich 0-1 m^2 .“
Die Fläche der Leuchte hat einen grossen Einfluss auf die totale Belastung. Es wird davon ausgegangen, dass die Fläche von Leuchten nicht grösser als 1 m^2 ist. Sollte dies aber tatsächlich der Fall sein, so kann die Inputkontrolle umgangen werden: Die Fläche der Leuchte sollte zu 1 m^2 gesetzt werden. Zusätzlich wird eine Zusatzmasse auf Höhe der Mastoberkante erstellt und dieser wird die fehlende Fläche zugewiesen.
- „Die Masse der Leuchte liegt nicht im Bereich 0-300 kg.“
Bei einer Masse von über 300 kg wird die Input-Zelle D17 als Flüchtigkeitsfehler bewertet.
- „Die Fläche einer Zusatzmasse liegt nicht im Bereich 0-1 m^2 .“
Wie bereits bei der Leuchte wird auch von Zusatzmassen eine Fläche unter 1 m^2 erwartet. Erneut kann die Inputkontrolle umgangen werden, indem man die Fläche auf mehrere Zusatzmassen auf der gleichen Höhe aufteilt.
- „Die Masse einer Zusatzmasse liegt nicht im Bereich 0-300 kg.“
Auch bei den Zusatzmassen werden Massen über 300 kg als Flüchtigkeitsfehler angesehen.
- „Bei der Berechnung können nur Zusatzmassen berücksichtigt werden, die sich nicht über der Mastspitze befinden.“
Hier wird auf eine Einschränkung des Programms aufmerksam gemacht. Es existieren Masten, deren Zusatzmassen über der eigentlichen Masthöhe liegen. Solche Zusatzmassen können in der aktuellen Version des Programms aber nicht berücksichtigt werden. Um die Weiterverwendung von falschen Resultaten zu verhindern, wird hier eine Fehlermeldung ausgegeben.

- „Die Breite einer Tür übersteigt 80% des Durchmessers auf dieser Höhe.“
Aus Erfahrungswerten ist bekannt, dass die Türbreiten nicht so grosse Werte annehmen. Dementsprechend sollte die Zeile 3 auf Fehler überprüft werden. Alternativ kann der Fehler auch beim Input der Mastdurchmesser bzw. -seitenlängen liegen.

6 Funktionsweise des Programms

Im Folgenden wird die Funktionsweise der Berechnung mit direkten Bezügen zum Code erläutert. Es wird dabei nicht auf konkrete Zeilennummern verwiesen, da sich diese durch jede kleine Anpassung des Programms verändern können. Stattdessen wird die Verbindung zum Code über Bilder von Code-Ausschnitten oder durch den Verweis auf Kommentare im Code herbeigeführt. Die referenzierten Kommentare lassen sich besonders ab ca. einem Drittel des Codes daran erkennen, dass sie im Gegensatz zu den restlichen Zeilen nicht eingerückt sind. Für das gesamte Programm gilt, dass die Variablennamen selbsterklärend sein sollten: Die Variable „anzahlelemente“ beispielsweise steht dementsprechend für die Anzahl Elemente, die vom Benutzer festgelegt und für die Berechnung verwendet werden.

Zusätzlich wird das Vorgehen und die Bedeutung von Variablen im Code selbst durch zahlreiche Kommentare bereits beschreiben. Dabei wird oft auf die Normen verwiesen, auf welchen die Berechnungen beruhen. Die Beschreibung „BA2023“ ist eine Abkürzung dafür, dass ein Beiwerk oder eine Gleichung aus der Bachelorarbeit „Berechnung der Windlasten von Beleuchtungsmasten“ [4] aus dem Frühjahrssemester 2023 stammt.

6.1 Programmstart: Bibliotheken und Umgang mit Excel-Dateien

„Import Bibliotheken“: Der Code beginnt mit dem Import von Bibliotheken wie „math“. Diese stellen verschiedene Funktionen zur Verfügung, welche für die Berechnung verwendet werden.

„Input-Datei einlesen und Darstellung Output vorbereiten“: Um den Input einzulesen, wird die Variable „Dateipfad“ erstellt. Weil sich das Python-Programm und die Input-Datei im gleichen Ordner befinden, genügt es nur den Namen der Input-Datei („Dateninput“) anzugeben. Ansonsten müsste tatsächlich der gesamte Dateipfad eingefügt werden. Anschliessend wird die Datei eingelesen und der Variable „daten“ zugewiesen. Es folgen print-Befehle. Diese sind im gesamten Programm dazu da, die Entwicklung des Programms zu unterstützen und Zwischenresultate zu kontrollieren. Sie haben nie eine tatsächliche Funktion für die schlussendliche Benutzung des Programms mittels Excel-Dateien. Um die Darstellung des Output zu ermöglichen, wird ein workbook mit dem Namen „sheet“ erstellt. Diesem wird der Titel „Resultate“ zugewiesen. Das wird der Name der Tabelle in der Output-Datei sein. Dann wird bereits „Ergebnisse Frequenzanalyse“ in die Zelle B2 geschrieben.

6.2 Benutzereingaben einlesen und kontrollieren

```
#Input-Parameter einlesen
#Welche Form hat der Mast? 1 = konisch (Hohlkreis), 2 = konisch (Achteck), 3 = zylindrisch (Hohlkreis)
form_excel = daten.iloc[1,1]
print("Form Excel:" , form_excel)
if form_excel == "konisch (Hohlkreis)":
    form = 1
elif form_excel == "konisch (Achteck)":
    form = 2
elif form_excel == "zylindrisch (Hohlkreis)":
    form = 3
```

Abbildung 6.1: Code-Ausschnitt, Einlesen der Mastform

„Input-Parameter einlesen“: Im Folgenden werden alle farbig markierten Zellen der Input-Datei von oben nach unten eingelesen und als Variable oder in einer Liste gespeichert. Die einzigen Zellen, die nicht eingelesen werden, sind die Angaben zur „Ausmitte“ von Leuchte und Zusatzmassen sowie die Norm, nach welcher die Lastbeiwerte bestimmt werden. Diese Informationen werden nur für das Programm „Standicherheit“, nicht aber für „Frequenzanalyse“ benötigt. Für die Mastform ist das Einlesen der Benutzereingabe folgendermassen implementiert (siehe Abbildung 6.1): Mit der Funktion `.iloc[1,1]` wird auf die Zelle B3 in der Input-Datei zugegriffen. Dabei wird zuerst die Zeile und danach die Spalte der Zelle definiert. Dass man mit `[1,1]` auf die Zelle B3 zugreift, lässt sich so erklären, dass die erste Zeile der Excel-Datei leer ist und deshalb nicht in „Daten“ enthalten ist. Zudem beginnt die Indexierung jeweils bei 0. Der Zelleninhalt wird der Variable „form_excel“ zugeordnet. Diese Variable wird nur in diesem Bereich des Codes verwendet. Nun wird eine neue Variable „form“ definiert, welche in Abhängigkeit von „form_excel“ einen anderen integer-Wert annimmt. „form“ wird später im Programm verwendet. Für die anderen Input-Werte ist das Vorgehen analog. Es gibt immer zuerst eine „_excel“-Variable, welcher der Zelleninhalt zugeordnet wird. Anschliessend wird eine Variable erstellt, die für die Berechnungen weiterverwendet wird. Da für den Durchmesser bzw. die Seitenlänge und die dazugehörige Höhe sowie für den Input der Zusatzmassen und Türen mehrere Werte möglich sind, wird der Input dort in Listen gespeichert. Die letzten eingelesenen Werte sind die gemessenen Eigenfrequenzen (siehe Abbildung 6.2). Hier wird zusätzlich die boolean-Variable „erfüllt“ definiert. Diese gibt an, ob der Benutzer eine gemessene Eigenfrequenz angegeben hat. Falls der Benutzer in die Zellen B54 und C54 eine 0 geschrieben hat, nimmt „erfüllt“ den Wert „True“ an. In diesem Fall wird der zweite Teil des Programms, die Berechnung der Steifigkeit der Einspannung und die erneute Bestimmung der statischen Deformation und der Eigenfrequenzen, nicht ausgeführt. Falls aber zumindest eine gemessene Eigenfrequenz eingegeben wird, nimmt „erfüllt“ den Wert „False“ an und das gesamte Programm wird durchlaufen. Zusätzlich muss dann festgelegt werden, ob die Steifigkeit der Einspannung mittels der ersten Eigenfrequenz in x- oder in y-Richtung berechnet werden soll. Dies zeigt die Variable „entscheidungsachse“ an: Falls die Eigenfrequenz in x-Richtung eingegeben wurde, wird sie für die Bestimmung verwendet („entscheidungsachse“ = „x“). Falls aber nur die Eigenfrequenz in y-Richtung bekannt ist, wird die Steifigkeit mithilfe der y-Richtung bestimmt.


```

#gemessene Eigenfrequenzen
f_1_gemessen_x = daten.iloc[52,1]
f_1_gemessen_y = daten.iloc[52,2]
print("1. Eigenfrequenz gemessen x:" , f_1_gemessen_x)
print("1. Eigenfrequenz gemessen y:" , f_1_gemessen_y)
erfüllt = False
if f_1_gemessen_x != 0:
    entscheidungsachse = "x"
elif f_1_gemessen_x == 0 and f_1_gemessen_y != 0:
    entscheidungsachse = "y"
else:
    erfüllt = True
    entscheidungsachse = "Keine"
print("Entscheidungsachse:" , entscheidungsachse)
print("erfüllt:" , erfüllt)

```

Abbildung 6.2: Code-Ausschnitt, Einlesen der gemessenen Eigenfrequenzen

```

#Berechnung inputabhängiger Variablen
#Berechnungen, jeweils in der Mitte eines Elements zwischen zwei "nodes" berechnet.
#Durchmesser wenn achteckiger Querschnitt
d_flanke = [0] * (len(d_list_eingabe))
d_aussen = [0] * (len(d_list_eingabe))
if form == 2:
    for i in range (len(d_list_eingabe)):
        #d_flanke bezeichnet den Flankendurchmesser bei achteckigen Querschnitten
        d_flanke[i] = d_list_eingabe[i]/math.tan(math.pi/8)
        print("Flankendurchmesser:" , d_flanke[i])
        #d_aussen bezeichnet den Aussendurchmesser bei achteckigen Querschnitten
        d_aussen[i] = d_list_eingabe[i]/(math.sin(math.pi/8))
        print("Aussendurchmesser" , d_aussen[i])
    else: #bei kreisförmigen Querschnitten sind d_flanke und d_aussen gleich
        d_flanke = d_list_eingabe.copy()
        d_aussen = d_list_eingabe.copy()

#Materialkennwerte
if material == 1:
    E = 210000*1000000 #in N/m^2
    rho = 7850 #in kg/m^3
elif material == 2:
    E = 210000*1000000 #in N/m^2
    rho = 7850 #in kg/m^3

```

Abbildung 6.3: Code-Ausschnitt, Bestimmung von Flanken- und Aussendurchmesser, f_y , E , ρ und γ_m

„Berechnung inputabhängiger Variablen“: Nachdem alle Benutzereingaben eingelesen wurden, wird für achteckige Querschnitte der Flanken- (Formel 6.1) und der Aussendurchmesser (Formel 6.2) berechnet (siehe Abbildung 6.3). Dabei bezeichnet der Flankendurchmesser den Abstand zwischen zwei parallel liegenden Flanken (Seiten), der Aussendurchmesser dagegen den Abstand von zwei gegenüberliegenden Ecken.

Flankendurchmesser

$$D_{Flanke} = \frac{L_{Seite}}{\tan(\pi/8)} \quad (6.1)$$

$$D_{Flanke} = \text{Flankendurchmesser} [m]$$

$$L_{Seite} = \text{Seitenlänge Achteck} [m]$$

Aussendurchmesser

$$D_{aussen} = \frac{L_{Seite}}{\sin(\pi/8)} \quad (6.2)$$

$$D_{aussen} = \text{Aussendurchmesser} [m]$$

$$L_{Seite} = \text{Seitenlänge Achteck} [m]$$

Bei rundem Querschnitt sind der vom Benutzer eingegeben Durchmesser, der Flanken- und der Aussendurchmesser identisch. Anschliessend werden abhängig vom Material des Mastes die Materialkennwerte f_y , E und ρ und der Widerstandsbeiwert γ_m definiert.

Es folgen weitere Berechnungen: Die Schwingungsdauer des Mastes wird gemäss der letztjährigen Bachelorarbeit [4] für die Bestimmung des Beiwerts β für das dynamische Verhalten konservativ als $T = 3.5s$ angenommen. Mithilfe der Schwingungsdauer folgt für β nach SN EN 40-3-1, 5.2.4 [3]:

Beiwert β

$$\beta = 1.0024 - 0.005 \cdot T^4 + 0.05144 \cdot T^3 - 0.22793 \cdot T^2 + 0.67262 \cdot T \quad (6.3)$$

$\beta = \text{Beiwert für das dynamische Verhalten von Masten} [-]$

$T = \text{Schwingungsdauer} [s]$

Der Beiwert δ wird nach SN EN 40-3-1, 5.2.3 [3] mithilfe der Masthöhe berechnet:

Beiwert δ

$$\delta = 1 - 0.01 \cdot h \quad (6.4)$$

$\delta = \text{Beiwert der Mastgrösse} [-]$

$h = \text{Masthöhe} [m]$

Im Folgenden wird der Topographiebeiwert f gemäss SN EN 40-3-1, 5.2.5 [3] zu 1 gesetzt. Für die Berechnung des Torsionswiderstandsmoments und der Biegemomentenwiderstände der Querschnitte mit einer Öffnung wird zu einem späteren Zeitpunkt die wirksame Länge der Türöffnung L benötigt. Dazu wird an dieser Stelle im Code eine Liste erstellt und die wirksame Länge für jede vorhandene Tür nach der Formel 6.5 aus SN EN 40-3-3, 5.6.2.2 [1] bestimmt:

wirksame Länge

$$L = a - 0.43 \cdot N \quad (6.5)$$

$L = \text{wirksame Länge der Türöffnung} [m]$

$a = \text{Gesamtlänge der Türöffnung} [m]$

$N = \text{Eckradius der Türöffnung} [m]$

Als nächstes wird der Referenzwert des Staudrucks q_{p0} entsprechend der Benutzereingabe definiert. Dann folgt die Definition der Luftdichte ρ_{Luft} , für welche gemäss dem Nationalen Anhang zu SN EN 1991-1-4 [5] $\rho_{Luft} = 1.2 \text{ kg/m}^3$ gilt. Für den Beiwert C_s zur Umrechnung von Überschreitungswahrscheinlichkeiten gilt in der Schweiz nach der Vorgängerarbeit [4]: $C_s = \sqrt{0.91}$. In ebendieser Arbeit wurde auch eine Formel für die über 10 min gemittelte Windgeschwindigkeit bestimmt:

Windgeschwindigkeit v_{ref}

$$v_{ref} = \sqrt{\frac{2 \cdot q_{p0} \cdot c_h(II, 10m)}{G_v(III, 10m)^2 \cdot \rho_{Luft}}} \quad (6.6)$$

$v_{ref} = \text{über 10 min gemittelte Windgeschwindigkeit} [m/s]$

$q_{p0} = \text{Referenzwert des Staudrucks} [N/mm^2]$

$\rho_{Luft} = \text{Luftdichte} [kg/m^3]$

$c_h(II, 10m) = \text{Profilbeiwert, Geländekategorie II, Höhe von 10 m} [-]$

$G_v(III, 10m) = \text{Böenfaktor, Geländekategorie III, Höhe von 10 m} [-]$

Dank der Formel 6.6 kann nun der Bezugswinddruck q_{10} gemäss SN EN 40-3-1, 5.2.2 berechnet werden:

Bezugswinddruck q_{10}

$$q_{10} = 0.5 \cdot \rho_{Luft} \cdot C_s^2 \cdot v_{ref}^2 \quad (6.7)$$

$q_{10} = \text{Bezugswinddruck} [N/m^2]$

$\rho_{Luft} = \text{Luftsdichte} [kg/m^3]$

$C_s = \text{Beiwert zur Umrechnung von Überschreitungswahrscheinlichkeiten} [-]$

$v_{ref} = \text{über 10 min gemittelte Windgeschwindigkeit} [m/s]$

Am Ende des Code-Abschnitts „Berechnung inputabhängiger Variablen“ werden abhängig von der Geländekategorie die Gradientenhöhe z_g und der Exponent der Bodenrauigkeit α_r (SIA 261, Tabelle 4 [2]) und abhängig von der Klasse der erlaubten horizontalen Verformung (SN EN 40-3-3, Tabelle 4 [1]) das Verhältnis zwischen Verformung und Masthöhe festgelegt.

```
#Inputkontrolle
inputkontrolle = True
#if hoehe > 20:
#    inputkontrolle = False
#    sheet.append(["Die verwendete Norm SN-EN-40-3-1 gilt nur bis zu einer Höhe von 20 m."])
for i in range(len(z_list)):
    if z_list[i] > hoehe:
        inputkontrolle = False
        sheet.append(["Der Durchmesser bzw. die Seitenlänge wurde für eine zu grosse Höhe eingegeben."])
    if d_list_eingabe[i] < 0.01 or d_list_eingabe[i] > 0.8:
        inputkontrolle = False
        sheet.append(["Der Durchmesser bzw. die Seitenlänge liegt nicht im Bereich 1-80 cm."])
for i in range(len(z_list)-1):
    if d_list_eingabe[i] < d_list_eingabe[i+1]:
        inputkontrolle = False
        sheet.append(["Der Durchmesser des Mastes nimmt nach oben zu anstatt ab."])
if t < 0.001:
    inputkontrolle = False
    sheet.append(["Die Wandstärke liegt unter 1 mm."])
```

Abbildung 6.4: Code-Ausschnitt, Inputkontrolle

Als nächstes wird die Inputkontrolle durchgeführt (siehe Abbildung 6.4). Diese wird im Kapitel Fehlermeldungen 5 detailliert erklärt. Falls der Input die Bedingungen des Abschnitts „Inputkontrolle“ nicht erfüllt, werden die im Folgenden erläuterten Berechnungen nicht ausgeführt. Stattdessen wird eine Beschreibung des Fehlers in die Output-Datei geschrieben. Oben in der Abbildung 6.4 ist ersichtlich, dass die Kontrolle der Masthöhe (Höhe nicht grösser als 20 m) auskommentiert ist. Falls man diese Bedingung einführen will, müssen nur die #-Zeichen entfernt werden.

6.3 Berechnung der Windlast und Flächenträgheitsmomente

```
#Beginn Berechnungen
if inputkontrolle == True:
    #z_node enthält die Höhe [m] des Nodes
    z_node = [None] * (anzahl_elemente+1)
    #c_h, Profilbeiwert c_h von jedem Node(gemäss Höhe des Nodes) gemäss SIA 261, 6.2.1.2
    c_h_node = [None] * (anzahl_elemente+1)
    #c_e, Standortbeiwert, gemäss BA2023, S.10
    c_e_node = [None] * (anzahl_elemente+1)
    #q_p [N/m^2], Staudruck gemäss SIA 261, 6.2.1.1
    q_p_node = [None] * (anzahl_elemente+1)
    #q_z [N/m^2], charakteristischer Winddruck gemäss SN-EN 40-3-1, 5.2.1
    q_z_node = [None] * (anzahl_elemente+1)
    #d_node_flanke, Flankendurchmesser, bei kreisförmigen Querschnitten gleich d_node_aussen
    d_node_flanke = [None] * (anzahl_elemente+1)
    #d_node_aussen, Aussendurchmesser, bei kreisförmigen Querschnitten gleich d_node_flanke
    d_node_aussen = [None] * (anzahl_elemente+1)
    #delta_q_h
```

Abbildung 6.5: Code-Ausschnitt, Beginn Berechnungen

Im Abschnitt „Beginn Berechnungen“ werden zuerst verschiedene Listen definiert. Die Bedeutung der einzelnen Listen ist inklusive Verweise auf die relevanten Normen durch Kommentare erklärt (siehe Abbildung 6.5). Das Programm modelliert den Mast durch einzelne diskrete Knoten (Englisch: „nodes“), welche durch „Elemente“ miteinander verbunden sind (siehe Abbildung 6.6). Der Benutzer kann in der Input-Datei die Anzahl Elemente wählen. Da der Mast in der Modellierung sowohl am oberen als auch am unteren Ende einen Knoten besitzt, ist die Anzahl Knoten um eins grösser als die eingegebene Anzahl der Elemente. Die Grösse der Listen entspricht jeweils der Anzahl Knoten.

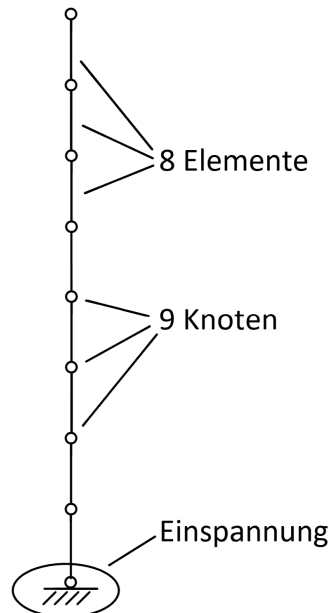


Abbildung 6.6: Darstellung Mastmodellierung mit 8 Elementen

Im Folgenden wird eine Schleife für jeden Knoten einmal durchlaufen. Als erstes

wird in dieser Schleife die mit z_{node} bezeichnete Höhe der Knoten bestimmt. Anschliessend wird in Funktion der Geländekategorie der Profilbeiwert c_h (SIA 261, 6.2.1.2 [2]) bestimmt:

Profilbeiwert c_h

$$c_h = 1.6 \cdot \left[\left(\frac{z}{z_g} \right)^{\alpha_r} + 0.375 \right]^2 \quad (6.8)$$

$c_h = \text{Profilbeiwert} [-]$

$z = \text{Höhe über dem Boden} [m]$

$z_g = \text{Gradientenhöhe} [m]$

$\alpha_r = \text{Exponent der Bodenrauigkeit} [-]$

Der Standortbeiwert $c_e(z)$ wird gemäss einer Formel aus der Bachelorarbeit des letzten Frühjahrssemester [4] berechnet:

Standortbeiwert $c_e(z)$

$$c_e(z) = c_h \cdot \frac{G_v(II, 10m)^2}{c_h(II, 10m)} \quad (6.9)$$

$c_e(z) = \text{Standortbeiwert} [-]$

$c_h = \text{Profilbeiwert des aktuell betrachteten Mastelements} [-]$

$G_v(III, 10m) = \text{Böenfaktor, Geländekategorie III, Höhe von 10 m} [-]$

$c_h(II, 10m) = \text{Profilbeiwert, Geländekategorie II, Höhe von 10 m} [-]$

Der nächste Schritt ist die Berechnung des charakteristischen Winddrucks q_z gemäss SN EN 40-3-1, 5.2.1 [3]:

charakteristischer Winddruck q_z

$$q_z = \delta \cdot \beta \cdot f \cdot c_e(z) \cdot q_{10} \quad (6.10)$$

$q_z = \text{charakteristischer Winddruck} [N/m^2]$

$\delta = \text{Beiwert der Mastgrösse} [-]$

$\beta = \text{Beiwert für das dynamische Verhalten von Masten} [-]$

$f = \text{Topographiebeiwert} [-]$

$c_e(z) = \text{Standortbeiwert} [-]$

$q_{10} = \text{Bezugswinddruck} [N/m^2]$

Anschliessend werden für jeden Knoten abhängig von der Mastform (zylindrisch oder konisch) die Flanken- und Aussendurchmesser berechnet. „delta_h_node“ entspricht dabei der Höhe des Mastabschnitts, welcher bei späteren Berechnungen jeweils einem Knoten zugeordnet wird. Mit Ausnahme des untersten und des obersten Knotens ist diese gleich der Höhe der Elemente. Danach wird die Windgeschwindigkeit auf Höhe des Elementmittelpunkts mittels Formel aus SN EN 40-3-1, 5.3.2 [3] ausgerechnet:

Windgeschwindigkeit v_{wind}

$$v_{\text{wind}} = \frac{1}{C_s} \cdot \sqrt{\frac{q_z}{0.5 \cdot \rho_{\text{Luft}} \cdot \delta \cdot \beta}} \quad (6.11)$$

$v_{\text{wind}} = \text{Windgeschwindigkeit} [m/s]$

$C_s = \text{Beiwert zur Umrechnung von Überschreitungswahrscheinlichkeiten} [-]$

$q_z = \text{charakteristischer Winddruck} [N/m^2]$

$\delta = \text{Beiwert der Mastgrösse} [-]$

$\beta = \text{Beiwert für das dynamische Verhalten von Masten } [-]$
 $\rho_{\text{Luft}} = \text{Luftdichte } [kg/m^3]$

Mithilfe der Windgeschwindigkeit folgt die Bestimmung der Reynoldszahl (SN EN 40-3-1, 5.3.2 [3]):

Reynoldszahl Re

$$Re = \frac{v_{\text{wind}} \cdot D}{\nu} \quad (6.12)$$

$Re = \text{Reynoldszahl } [-]$

$v_{\text{wind}} = \text{Windgeschwindigkeit } [m/s]$

$D = \text{Mastdurchmesser bzw. Flankendurchmesser } [m]$

$\nu = \text{kinematische Viskosität der Luft: } \nu = 15.1 \cdot 10^{-6} m/s^2$

Daraufhin wird die Reynoldszahl verwendet, um gemäss SN EN 40-3-1, Bild 3 [3] den Formbeiwert c zu ermitteln. Im Folgenden wird für jeden Knoten beziehungsweise den zugehörigen Mastabschnitt die totale wirksame Windfläche, betrachtet aus den beiden Richtungen x und y , bestimmt. Die Flächen ergeben sich aus den Formbeiwerten multipliziert mit „delta_h_node“ und dem Durchmesser auf Höhe des Knotens (siehe Abbildung 6.7, oberste Zeile). Zusätzlich werden die projizierten Flächen der Zusatzmassen und der Leuchte dem nächsten Knoten zugeordnet. Da allgemein nichts über die Form der Zusatzmassen bekannt ist, wird für sie ebenso wie für die Leuchte ein Formbeiwert von 1 verwendet. Falls sich Zusatzmassen genau in der Mitte zwischen zwei Knoten befinden, werden die projizierten Flächen gleichmässig auf die beiden Knoten aufgeteilt.

```
flaeche_mast_node[i] = delta_h_node[i]*d_node_aussen[i]*c_formbeiwert_node[i]
print("Fläche Mastabschnitt:", flaeche_mast_node[i]) #Kontrollausgabe ohne Funktion
for k in range(len(zusatzmasse_list)): #Falls eine Zusatzmasse gerade in der Mitte zwischen zwei Nodes liegt, wird sie gleichmässig
    if i == 0 and z_zusatzmasse_list[k] < delta_h_node[i]:
        flaeche_zm_node_x[i] += A_x_zusatzmasse_list[k]
        flaeche_zm_node_y[i] += A_y_zusatzmasse_list[k]
    elif i == 0 and z_zusatzmasse_list[k] == delta_h_node[i]:
        flaeche_zm_node_x[i] += 0.5*A_x_zusatzmasse_list[k]
        flaeche_zm_node_y[i] += 0.5*A_y_zusatzmasse_list[k]
    elif i == anzahl_elemente and z_zusatzmasse_list[k] <= hoehe and z_zusatzmasse_list[k] > (hoehe-delta_h_node[i]):
        flaeche_zm_node_x[i] += A_x_zusatzmasse_list[k]
        flaeche_zm_node_y[i] += A_y_zusatzmasse_list[k]
    elif i == anzahl_elemente and z_zusatzmasse_list[k] == (hoehe-delta_h_node[i]):
        flaeche_zm_node_x[i] += 0.5*A_x_zusatzmasse_list[k]
        flaeche_zm_node_y[i] += 0.5*A_y_zusatzmasse_list[k]
    elif z_node[i]-(delta_h_node[i]/2) < z_zusatzmasse_list[k] and z_node[i]+(delta_h_node[i]/2) > z_zusatzmasse_list[k]:
        flaeche_zm_node_x[i] += A_x_zusatzmasse_list[k]
        flaeche_zm_node_y[i] += A_y_zusatzmasse_list[k]
    elif z_node[i]-(delta_h_node[i]/2) == z_zusatzmasse_list[k] or z_node[i]+(delta_h_node[i]/2) == z_zusatzmasse_list[k]:
        flaeche_zm_node_x[i] += 0.5*A_x_zusatzmasse_list[k]
        flaeche_zm_node_y[i] += 0.5*A_y_zusatzmasse_list[k]
#Leuchte dazurechnen
if i == anzahl_elemente:
    flaeche_zm_node_x[i] += A_x_leuchte
    flaeche_zm_node_y[i] += A_y_leuchte
flaeche_tot_node_x[i] = flaeche_mast_node[i] + flaeche_zm_node_x[i]
flaeche_tot_node_y[i] = flaeche_mast_node[i] + flaeche_zm_node_y[i]
print("Fläche total, x-Richtung:", flaeche_tot_node_x[i])
print("Fläche total, y-Richtung:", flaeche_tot_node_y[i])
#Windkraft pro Node, auf charakteristischem Niveau (für Verformungen):
f_d_node_x[i] = flaeche_tot_node_x[i]*q_z_node[i]
f_d_node_y[i] = flaeche_tot_node_y[i]*q_z_node[i]
```

Abbildung 6.7: Code-Ausschnitt, Fläche und Windlast pro Knoten

Am Ende dieser Schleife werden die pro Knoten wirkenden Windlasten bei Wind aus x - bzw. y -Richtung ermittelt. Sie ergeben sich durch die Multiplikation der wirksamen Windfläche mit dem charakteristischen Winddruck q_z auf der entsprechenden Höhe (siehe Abbildung 6.7, ganz unten).

```

#Berechnungen Schleife 2
#z_m enthält die Höhe [m] des Mittelpunktes von jedem Element
z_m = [0] * (anzahl_elemente)
#d_m_flanke, Flankendurchmesser, bei kreisförmigen Querschnitten gleich d_m_aussen
d_m_flanke = [0] * (anzahl_elemente)
#d_m_aussen, Aussendurchmesser, bei kreisförmigen Querschnitten gleich d_m_flanke
d_m_aussen = [0] * (anzahl_elemente)
#d_seite_elementmitte, Seitenlänge aussen in der Mitte des Elements, wird nur für achteckige Querschnitte verwendet
d_seite_elementmitte = [0] * (anzahl_elemente)
#A, Querschnittsfläche in der Mitte eines Elements
A = [0] * (anzahl_elemente)
#L, Länge eines Elements
L = hoehe/anzahl_elemente
#teilt die Zusatzmassen dem nächsten node zu, falls die Zusatzmasse genau in der Mitte liegt, wird sie aufgeteilt
masse_node_liste = [0] * (anzahl_elemente+1)
#r_m_elementmitte, mittlerer Radius des Querschnitts in der Elementmitte
r_m_elementmitte = [0] * (anzahl_elemente)
#theta, halber Türöffnungswinkel gemäss SN-EN 40-3-3
theta = [0] * (anzahl_elemente)

```

Abbildung 6.8: Code-Ausschnitt, Berechnungen Schleife 2

Der Code-Abschnitt „Berechnungen Schleife 2“ führt eine Schleife für jedes Element einmal aus. Damit wird diese Schleife einmal weniger durchlaufen als die vorherige. Es ist auch erkennbar, dass die in der Abbildung 6.8 definierten Listen dementsprechend eine kleinere Dimension aufweisen als die der Abbildung 6.5. Die Schleife beginnt mit der Bestimmung der Höhe „z_m“ jedes Elementmittelpunktes. Zusätzlich werden auf diesen Höhen jeweils Flankendurchmesser, Aussendurchmesser, mittlerer Radius und bei achteckigen Querschnitten auch die Seitenlänge bestimmt. Es folgt die in Abbildung 6.9 gezeigte Berechnung der Querschnittsfläche. Anschließend wird die Liste „masse_node_liste“ behandelt. Dabei werden die Massen der vom Benutzer eingegebenen Zusatzmassen auf die am nächsten liegenden Knoten (nicht Elemente) verteilt. Liegen sie genau in der Mitte zwischen zwei Knoten, werden sie aufgeteilt. Die Masse der Leuchte wird in diesem Schritt nicht berücksichtigt. Das geschieht zu einem späteren Zeitpunkt. Es ist insbesondere wichtig zu verstehen, dass es sich hier nur um die Zusatzmassen handelt und nicht auch um die Masse des Mastes selbst.

```

if form == 1 or form == 3:
    A[i] = (d_m_aussen[i]**2-(d_m_aussen[i]-2*t)**2)*math.pi/4
elif form == 2:
    A[i] = 8*t*(d_seite_elementmitte[i]-t/math.tan(3*math.pi/8))
print("Fläche:", A[i])

for k in range(len(zusatzmasse_liste)): #Falls eine Zusatzmasse gerade in der Mitte eines Elements liegt, wird sie gleichmässig auf
    if i == 0 and z_zusatzmasse_list[k] == 0:
        masse_node_liste[i] += m_zusatzmasse_list[k]
    elif i == anzahl_elemente and z_zusatzmasse_list[k] == hoehe: #damit Zusatzmassen an der Spitze sicher auch berücksichtigt werden
        masse_node_liste[i+1] += m_zusatzmasse_list[k]
    elif z_m[i]-L < z_zusatzmasse_list[k] and z_m[i] > z_zusatzmasse_list[k]:
        masse_node_liste[i] += m_zusatzmasse_list[k]
    elif z_m[i]-L == z_zusatzmasse_list[k] or z_m[i] == z_zusatzmasse_list[k]:
        masse_node_liste[i] += 0.5*m_zusatzmasse_list[k]
    if i == anzahl_elemente: #für den Fall, dass eine Masse auf die beiden obersten nodes aufgeteilt werden muss
        masse_node_liste[i+1] += 0.5*m_zusatzmasse_list[k]

```

Abbildung 6.9: Code-Ausschnitt, Querschnittsfläche und Zusatzmassen

Es folgt eine Berücksichtigung möglicher Türen im Mast. Falls der aktuell betrachtete Querschnitt eine Öffnung enthält, wird der halbe Türöffnungswinkel θ mithilfe der Türbreite und des Querschnittsradius bzw. -durchmessers berechnet. Anschließend werden, wie in Abbildung 6.10 dargestellt, abhängig von der Geometrie des Querschnitts die Abstände m_x , m_y , m_{0x} und m_{0y} berechnet. Diese werden gemäss SN EN 40-3-3, 5.6.2.3.2 [1] folgendermassen definiert:

m_x = Abstand von der Mastwandmitte am Rand der Türöffnung zur x-Achse [m]
 m_y = Abstand von der Mastwandmitte am Rand der Türöffnung zur y-Achse [m]
 m_{0x} = Abstand vom Schwerpunkt der Türverstärkung zur x-Achse [m]

m_{0y} = Abstand vom Schwerpunkt der Türverstärkung zur y-Achse [m]

Diese Abstände dienen allesamt als Hilfsgrößen für die Berechnung der Flächenträgheitsmomente. Dass sie separat berechnet und aufgeführt werden, lässt sich auf die simultane Entwicklung des Programms „Standicherheit“, in welchem diese Abstände eine grössere Bedeutung aufweisen, zurückführen.

```

if form == 2:
    if breite_tuere_list[a] <= d_seite_elementmitte[i]:
        m_x[i] = 0.5*d_seite_elementmitte[i]*(1+math.sqrt(2)) - 0.5*t
        m_y[i] = 0.5*breite_tuere_list[a]
    elif breite_tuere_list[a] > d_seite_elementmitte[i]:
        m_x[i] = 0.5*d_seite_elementmitte[i]*(1+math.sqrt(2)) - 0.5*(breite_tuere_list[a]-d_seite_elementmitte[i]) - math.sqrt(2)/4*t
        m_y[i] = 0.5*breite_tuere_list[a] - math.sqrt(2)/4*t
    if breite_tuere_list[a] + 2*breite_verstaerkung_list[a] <= d_seite_elementmitte[i]:
        m_0x[i] = 0.5*d_seite_elementmitte[i]*(1+math.sqrt(2)) - t - 0.5*dicke_verstaerkung_list[a]
        m_0y[i] = m_y[i] + 0.5*breite_verstaerkung_list[a]
    elif breite_tuere_list[a] <= d_seite_elementmitte[i]:
        m_0x[i] = 0.5*d_seite_elementmitte[i]*(1+math.sqrt(2)) - (t+0.5*dicke_verstaerkung_list[a]+0.5*breite_verstaerkung_list[a])*math.sqrt(2)
        m_0y[i] = 0.5*d_seite_elementmitte[i] - (t+0.5*dicke_verstaerkung_list[a])*math.sqrt(2)/2 + breite_verstaerkung_list[a]*math.sqrt(2)
    elif breite_tuere_list[a] > d_seite_elementmitte[i]:
        m_0x[i] = 0.5*d_seite_elementmitte[i]*(1+math.sqrt(2)) - 0.5*(breite_tuere_list[a]-d_seite_elementmitte[i]) - (t+0.5*dicke_verstaerkung_list[a])*math.sqrt(2)/2 + breite_verstaerkung_list[a]*math.sqrt(2)/4
        m_0y[i] = 0.5*breite_tuere_list[a] - (t+0.5*dicke_verstaerkung_list[a])*math.sqrt(2)/2 + breite_verstaerkung_list[a]*math.sqrt(2)/4
if form == 1 or form == 3:
    m_x[i] = r_elementmitte[i]*math.cos(theta[i])
    m_y[i] = r_elementmitte[i]*math.sin(theta[i])
    m_0x[i] = m_x[i] - 0.5*breite_verstaerkung_list[a]*math.sin(theta[i]) - 0.5*(t+dicke_verstaerkung_list[a])*math.cos(theta[i])
    m_0y[i] = m_y[i] - 0.5*(t+dicke_verstaerkung_list[a])*math.sin(theta[i]) + 0.5*breite_verstaerkung_list[a]*math.cos(theta[i])

```

Abbildung 6.10: Code-Ausschnitt, Bestimmung von m_x , m_y , m_{0x} und m_{0y}

In einem letzten Schritt erfolgt die Berechnung der Flächenträgheitsmomente I_x und I_y aller Querschnitte (der Elementmitten). Hierbei werden die verschiedenen Arten von Geometrien (Achteck oder Hohlkreis als Querschnitt, Türbreite grösser oder kleiner als Seitenlänge, Öffnungen unverstärkt oder verstärkt etc.) unterschieden. Die Formeln der einzelnen Fälle können dem Code entnommen werden. Bevor diese grosse zweite Schleife beendet wird, wird die Ausrichtung der Öffnung (falls vorhanden) überprüft. Da die bisher aufgeführten Formeln für eine x-Ausrichtung gelten, werden die Werte für I_x und I_y vertauscht, falls die Tür tatsächlich in y-Richtung ausgerichtet ist.

6.4 Statische Deformation und Eigenfrequenzen bei starrer Einspannung

Der in diesem Unterkapitel 6.4 beschriebene Code lässt sich in drei Abschnitte einteilen. Den Beginn macht die „Mastmodellierung“ mittels Steifigkeits- und Massmatrix. Diese werden anschliessend verwendet um in einem zweiten Teil die statische Deformationen und schlussendlich die Eigenmodi und die entsprechenden Eigenfrequenzen zu ermitteln. Die Mastmodellierung wird jeweils zweidimensional durchgeführt (siehe Abbildung 6.11). Das bedeutet, dass jeder Knoten 3 Freiheitsgrade aufweist: Translation in Richtung der Mastachse, Translation senkrecht dazu in der Ebene, und Rotation um eine Achse senkrecht zur Ebene. Da wir das Schwingungsverhalten in y- und in x-Richtung betrachten wollen, wird die Modellierung wie abgebildet zweimal durchgeführt. Der gesamte Code dieses Unterkapitels 6.4 wird dementsprechend zuerst für die x-Richtung und anschliessend für die y-Richtung durchlaufen (siehe Abbildung 6.12).

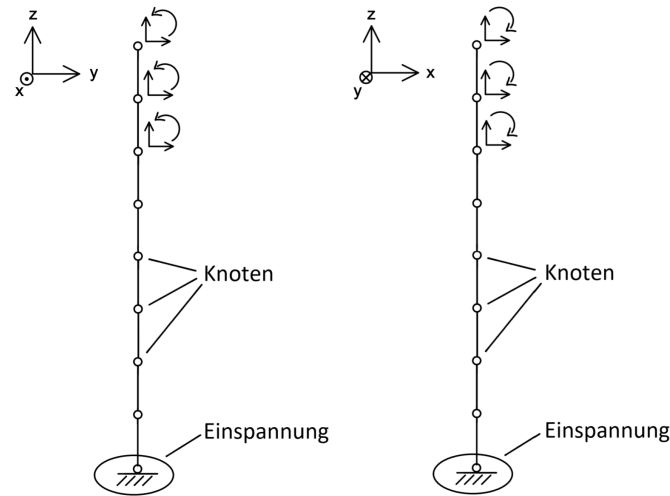


Abbildung 6.11: Darstellung zweidimensionale Mastmodellierung

```
#Mastmodellierung
#die folgende Berechnung wird 2 mal durchgeführt: zuerst um die x-Achse, danach um die y-Achse
for achse in range (2):
    #I, Flächenträgheitsmoment um die x-Achse oder um die y-Achse, je nach Berechnungsrichtung
    I = [0] * (anzahl_elemente)
    for i in range (anzahl_elemente):
        if achse == 0:
            I[i] = I_x[i]
        if achse == 1:
            I[i] = I_y[i]
    anzahl_nodes = anzahl_elemente + 1
    anzahl_dof = 3*anzahl_nodes
    k_matrix_1 = np.zeros( (anzahl_dof,anzahl_dof) )
    m_matrix_1 = np.zeros( (anzahl_dof,anzahl_dof) )
    for i in range (anzahl_elemente):
        k_1a = np.array( [ [E*A[i]/L, 0, 0],
                           [0, 12*E*I[i]/(L**3), 6*E*I[i]/(L**2)],
                           [0, 6*E*I[i]/(L**2), 4*E*I[i]/L ] ] )
        k_1b = np.array( [ [-E*A[i]/L, 0, 0],
                           [0, -12*E*I[i]/(L**3), 6*E*I[i]/(L**2)],
                           [0, 6*E*I[i]/(L**2), 2*E*I[i]/L ] ] )
        k_1c = np.array( [ [-E*A[i]/L, 0, 0],
                           [0, -12*E*I[i]/(L**3), -6*E*I[i]/(L**2)],
                           [0, 6*E*I[i]/(L**2), 2*E*I[i]/L ] ] )
        k_1d = np.array( [ [E*A[i]/L, 0, 0],
                           [0, 12*E*I[i]/(L**3), -6*E*I[i]/(L**2)],
                           [0, -6*E*I[i]/(L**2), 4*E*I[i]/L ] ] )

    k_matrix_3_dof = np.array( [ [k_1a[0][0], k_1a[0][1], k_1a[0][2], k_1b[0][0], k_1b[0][1], k_1b[0][2] ],
                                  [k_1a[1][0], k_1a[1][1], k_1a[1][2], k_1b[1][0], k_1b[1][1], k_1b[1][2] ],
                                  [k_1a[2][0], k_1a[2][1], k_1a[2][2], k_1b[2][0], k_1b[2][1], k_1b[2][2] ],
                                  [k_1c[0][0], k_1c[0][1], k_1c[0][2], k_1d[0][0], k_1d[0][1], k_1d[0][2] ],
                                  [k_1c[1][0], k_1c[1][1], k_1c[1][2], k_1d[1][0], k_1d[1][1], k_1d[1][2] ],
                                  [k_1c[2][0], k_1c[2][1], k_1c[2][2], k_1d[2][0], k_1d[2][1], k_1d[2][2] ] ] )
```

Abbildung 6.12: Code-Ausschnitt, Beginn Mastmodellierung

Im ersten Programmabschnitt „Mastmodellierung“ wird zuerst festgelegt, ob das Flächenträgheitsmoment I_x und I_y verwendet werden muss. Zusätzlich wird die Anzahl Knoten und damit auch die der Freiheitsgrade definiert, da diese die Dimensionen der Matrizen vorgeben. Dann beginnt die Modellierung mit der Erstellung der Masse- und Steifigkeitsmatrix für jeweils ein Element ([6], S.4) :

$$\underline{\underline{K}} = \begin{bmatrix} EA/L & 0 & 0 & -EA/L & 0 & 0 \\ 0 & 12EI/L^3 & 6EI/L^2 & 0 & -12EI/L^3 & 6EI/L^2 \\ 0 & 6EI/L^2 & 4EI/L & 0 & -6EI/L^2 & 2EI/L \\ -EA/L & 0 & 0 & EA/L & 0 & 0 \\ 0 & -12EI/L^3 & -6EI/L^2 & 0 & 12EI/L^3 & -6EI/L^2 \\ 0 & 6EI/L^2 & 2EI/L & 0 & -6EI/L^2 & 4EI/L \end{bmatrix}$$

$$\underline{\underline{M}} = \rho \cdot A \cdot L \begin{bmatrix} 1/3 & 0 & 0 & 1/6 & 0 & 0 \\ 0 & 13/35 & 11L/210 & 0 & 9/70 & -13L/420 \\ 0 & 11L/210 & L^2/105 & 0 & 13L/420 & -L^2/140 \\ 1/6 & 0 & 0 & 1/3 & 0 & 0 \\ 0 & 9/70 & 13L/420 & 0 & 13/35 & -11L/210 \\ 0 & -13L/420 & -L^2/140 & 0 & -11L/210 & L^2/105 \end{bmatrix}$$

Dabei beziehen sich A und I jeweils auf den Querschnitt in der Mitte eines Elements, E und ρ sind Materialeigenschaften und L ist die Elementlänge.

```

for j in range(6):
    for q in range(6):
        k_matrix_1[3*i+j][3*i+q] += k_matrix_3_dof[j][q]
        m_matrix_1[3*i+j][3*i+q] += m_matrix_3_dof[j][q]
print("k matrix_1 ohne Randbedingungen:")
print(k_matrix_1)
print("m matrix_1 ohne Randbedingungen:")
print(m_matrix_1)

#Zusatzmassen und Leuchte an der Spitze hinzufügen
#Zusatzmassen
print("Extramassen:" , masse_node_liste)
for i in range(anzahl_nodes):
    m_matrix_1[3*i][3*i] += masse_node_liste[i]
    m_matrix_1[3*i+1][3*i+1] += masse_node_liste[i]
#Leuchte
m_matrix_1[anzahl_dof-3][anzahl_dof-3] += m_leuchte
m_matrix_1[anzahl_dof-2][anzahl_dof-2] += m_leuchte

```

Abbildung 6.13: Code-Ausschnitt, Zusammenstellung System-Matrizen

In Abbildung 6.13 wird gezeigt, wie aus den einzelnen Elementmatrizen eine Steifigkeits- und Massematrix für den gesamten Mast zusammengefügt wird. Die Dimensionen der quadratischen Matrizen ergeben sich aus der Anzahl Knoten mal 3 (Anzahl Freiheitsgrade pro Knoten). Dabei wird die Einspannung noch nicht berücksichtigt. Dafür werden, wie ebenfalls in Abbildung 6.13 ersichtlich, die Massen der Leuchte und Zusatzmassen zur Massematrix hinzugefügt. Dazu wird bei den Zusatzmassen die im Unterkapitel 6.3 erläuterte „masse_node_liste“ verwendet. Danach wird ein Kraftvektor gebildet (siehe Abbildung 6.14). Die Dimension des Vektors stimmt mit der Anzahl Freiheitsgrade des gesamten Systems überein. Für die Freiheitsgrade der Rotation und der Translation in Richtung der Mastachse sind die Einträge 0 und für die Freiheitsgrade der Translation senkrecht zur Mastachse entsprechen die Einträge den pro Knoten wirkenden Windlasten.

```

#Kraftvektor (nur Windkraft) erstellen
f_vektor_1 = np.zeros(anzahl_dof)
for i in range (anzahl_nodes):
    if achse == 0:
        f_vektor_1[3*i+1] = f_d_node_x[i]
    if achse == 1:
        f_vektor_1[3*i+1] = f_d_node_y[i]
print("Kraftvektor F ohne Randbedingungen:" , f_vektor_1)

#dofs, die durch einspannung gehalten werden, löschen
dof_gehalten = [0,1,2] #mit 2: biegesteif eingespannt
anzahl_dof_gehalten = len(dof_gehalten)
m_matrix = np.zeros( (anzahl_dof-anzahl_dof_gehalten,anzahl_dof-anzahl_dof_gehalten) )
k_matrix = np.zeros( (anzahl_dof-anzahl_dof_gehalten,anzahl_dof-anzahl_dof_gehalten) )
f_vektor = np.zeros(anzahl_dof-anzahl_dof_gehalten)
a = 0
for i in range (anzahl_dof):
    b = 0
    if i not in dof_gehalten:
        f_vektor[a] = f_vektor_1[i]
        for j in range (anzahl_dof):
            if j not in dof_gehalten:
                m_matrix[a][b] = m_matrix_1[i][j]
                k_matrix[a][b] = k_matrix_1[i][j]
                b += 1
        a += 1

```

Abbildung 6.14: Code-Ausschnitt, Kraftvektor, Berücksichtigung der Einspannung

Durch die (momentan noch perfekt biegesteife) Einspannung sind die drei Freiheitsgrade des ersten Knoten blockiert. Deshalb werden sie in Abbildung 6.14 aus den beiden Matrizen und dem Kraftvektor entfernt.

```

#statische Deformation berechnen
u_statisch = np.zeros(anzahl_dof-anzahl_dof_gehalten)
k_matrix_invers = np.linalg.inv(k_matrix)
u_statisch = np.dot(k_matrix_invers,f_vektor)
print("u_statisch:" , u_statisch)
u_gross = np.zeros(anzahl_dof)
c = 0
for i in range (anzahl_dof):
    if i in dof_gehalten:
        u_gross[i] = 0
        c += 1
    else:
        u_gross[i] = u_statisch[i-c]
print("u_gross:" , u_gross)
u_horizontal = np.zeros(anzahl_nodes)
for i in range (anzahl_nodes):
    u_horizontal[i] = u_gross[3*i+1]
print("u_horizontal:" , u_horizontal)
if achse == 0:
    print("zulässige horizontale Verschiebung:" , round(verschiebung_zugelassen*hoehe*1000,2) , "mm")
    print("Die horizontale Verschiebung der Mastspitze in x-Richtung ist:" , round(u_horizontal[anzahl_nodes-1]*1000,2) , "mm")
    sheet.append(["zulässige horizontale Verschiebung [mm]:" , round(verschiebung_zugelassen*hoehe*1000,2) ])
    sheet.append(["horizontale Verschiebung, x-Richtung [mm]:" , round(u_horizontal[anzahl_nodes-1]*1000,2) ])
if achse == 1:
    print("zulässige horizontale Verschiebung:" , round(verschiebung_zugelassen*hoehe*1000,2) , "mm")
    print("Die horizontale Verschiebung der Mastspitze in y-Richtung ist:" , round(u_horizontal[anzahl_nodes-1]*1000,2) , "mm")

```

Abbildung 6.15: Code-Ausschnitt, Berechnung der statischen Deformationen

In Abbildung 6.15 ist zu sehen, wie der Vektor „u.horizontal“, welcher die horizontale Verschiebung des Mastes für den Grenzzustand der Gebrauchstauglichkeit enthält, berechnet wird. Dazu wird die folgende Gleichung gelöst ([7], S.36):

Verschiebung $\underline{u}$

$$\underline{u} = \underline{K}_{sys}^{-1} \cdot \underline{f} \quad (6.13)$$

$\underline{u}$ = Verschiebungsvektor der ungehaltenen Freiheitsgrade

$\underline{K}_{sys}^{-1}$ = inverse Systemsteifigkeitsmatrix für die ungehaltenen Freiheitsgrade

$\underline{f}$ = Kraftvektor der ungehaltenen Freiheitsgrade, enthält die Windlasten

Die Resultate werden auch sogleich in das workbook „sheet“ geschrieben. Zusätzlich wird ein Plot des verformten Mastes erstellt und hinzugefügt.

```

#Eigenfrequenzen und -modi berechnen
D, V = np.linalg.eig(np.dot(np.linalg.inv(m_matrix), k_matrix))
#D = eigenwerte, V = eigenvektoren
print("D:" , D)
print("V:" , V) #eigenmodi sind Spalten in der V-matrix
sortiert_D = sorted(D)
print("sortiert_D:")
print(sortiert_D)
transponiert_V = np.transpose(V) #jetzt sind die eigenmodi Zeilen in der transponierten V-matrix
print("transponiert_V:")
print(transponiert_V)
sortiert_V = np.zeros( (len(D),len(D)) )
for i in range (len(D)):
    for j in range (len(D)):
        if sortiert_D[i] == D[j]:
            sortiert_V[i] = transponiert_V[j]
print("sortiert_V:")
print(sortiert_V)
normalisiert_V = np.zeros( (len(D),len(D)) )
for i in range (len(D)):
    max_verschiebung = max(sortiert_V[i], key=abs)
    if max_verschiebung != 0:
        ratio = 1/max_verschiebung
        ratio = abs(ratio)
        normalisiert_V[i] = [x*ratio for x in sortiert_V[i]]
    else:
        normalisiert_V[i] = sortiert_V[i]
print("normalisiert_V:")
print(normalisiert_V)
eigenfrequenzen = sortiert_D
eigenfrequenzen = [math.sqrt(x)/(2*math.pi) for x in eigenfrequenzen]

```

Abbildung 6.16: Code-Ausschnitt, Berechnung Eigenfrequenzen und -modi

Die Eigenmodi und die Kreisfrequenzen ω können durch das Lösen eines Eigenwertproblems bestimmt werden. In der Abbildung 6.16 (oben) löst Python die folgende Gleichung ([8], S.183):

$$(\underline{K}_{sys} - \underline{M}_{sys} \cdot \omega^2) \cdot \underline{x} = 0 \quad (6.14)$$

$\underline{K}_{sys}$ = Systemsteifigkeitsmatrix für die ungehaltenen Freiheitsgrade

$\underline{M}_{sys}$ = Massematrix für die ungehaltenen Freiheitsgrade

ω = Kreisfrequenz bzw. Winkelgeschwindigkeit der Schwingung [rad/s]

$\underline{x}$ = Eigenmodus, Auslenkung bzw. Verdrehung der Knoten

Dabei wird pro ungehaltener Freiheitsgrad des Systems eine Kreisfrequenz sowie ein Eigenmodus bestimmt. Die Kreisfrequenzen ω werden in einem Vektor und die Eigenmodi werden in Matrixform gespeichert. Die Eigenmodi werden zuerst transponiert, sodass eine Zeile der Matrix einem Eigenmodus entspricht und anschließend normalisiert um später eine schönere Darstellung zu erlauben. Des Weiteren werden die Eigenfrequenzen des Mastes berechnet ([9], S.14):

$$f = \frac{\omega}{2 \cdot \pi} \quad (6.15)$$

f = Frequenz der Schwingung, entspricht der Eigenfrequenz [Hz]

ω = Kreisfrequenz bzw. Winkelgeschwindigkeit der Schwingung [rad/s]

Anschließend werden die gehaltenen Freiheitsgrade der Einspannung zu den Eigenmodi hinzugefügt (mit den Beträgen 0). Die ersten fünf Eigenfrequenzen werden zum workbook „sheet“ hinzugefügt und die dazugehörigen Eigenmodi werden ge-

plottet. Damit ist der Teil des Programms beendet, welcher auch ohne gemessene Eigenfrequenzen ausgeführt werden kann.

6.5 Berechnung der Steifigkeit der Einspannung

```
#iterative Berechnung der Steifigkeit der Einspannung
if erfüllt == False:
    k_bodenfeder = 10000 #in [Nm/rad]
    anzahl_iterationen = 0
    if entscheidungsachse == "x":
        achse = 0
        f_1_gemessen = f_1_gemessen_x
    else:
        achse = 1
        f_1_gemessen = f_1_gemessen_y
    print("Entscheidungsachse:", entscheidungsachse)
    print("erfüllt:", erfüllt)
    veraenderung = 1000000 #um so viel [Nm/rad] wird der Wert für k_bodenfeder verändert, später halbiert sich die Grösse der Veränderungen.
    groesser = True #gibt an, in welche Richtung der Wert von k_bodenfeder im vorherigen Schritt verändert wurde
    for q in range(50):
        print("Iteration wird durchgeführt.")
        anzahl_iterationen += 1
        print("Achse: ", achse)
        for i in range(anzahl_elemente):
            if achse == 0:
                I[i] = I_x[i]
            if achse == 1:
                I[i] = I_y[i]
        anzahl_nodes = anzahl_elemente + 1
        anzahl_dof = 3*anzahl_nodes
        k_matrix_1 = np.zeros( (anzahl_dof,anzahl_dof) )
        m_matrix_1 = np.zeros( (anzahl_dof,anzahl_dof) )
```

Abbildung 6.17: Code-Ausschnitt, Bestimmung der Steifigkeit der Einspannung

Im nächsten Code-Abschnitt wird die Steifigkeit der Einspannung bestimmt. Mit der Variable „erfüllt“ (siehe Abbildung 6.17, ganz oben) wird überprüft, dass der Benutzer eine gemessene Eigenfrequenz eingegeben hat, welche für die Berechnung verwendet werden kann. Zusätzlich wird definiert, ob die Eigenfrequenz in x- oder y-Richtung für die Bestimmung massgebend ist. Die Berechnung verläuft iterativ. Es wird jeweils ein Wert für die Steifigkeit angenommen, dann werden damit die Eigenfrequenzen bestimmt und die erste Eigenfrequenz wird mit dem Messresultat verglichen. Die Bestimmung der Eigenfrequenzen geschieht wieder durch eine Modellierung wie in Unterkapitel 6.4. Der einzige Unterschied ergibt sich beim Rotationsfreiheitsgrad des untersten Knotens. Bisher wurde dieser als gehalten betrachtet und entfernt. Nun gehört er zu den ungehaltenen Freiheitsgraden. In der Steifigkeitsmatrix wird dafür die Steifigkeit der Einspannung addiert (siehe Abbildung 6.18, ganz oben).

```
k_matrix_1[2][2] += k_bodenfeder

#dofs, die durch Einspannung gehalten werden, Löschen
dof_gehalten = [0,1] #mit 2: biegesteif eingespannt
anzahl_dof_gehalten = len(dof_gehalten)
m_matrix = np.zeros( (anzahl_dof-anzahl_dof_gehalten,anzahl_dof-anzahl_dof_gehalten) )
k_matrix = np.zeros( (anzahl_dof-anzahl_dof_gehalten,anzahl_dof-anzahl_dof_gehalten) )
f_vektor = np.zeros(anzahl_dof-anzahl_dof_gehalten)
a = 0
for i in range(anzahl_dof):
    b = 0
    if i not in dof_gehalten:
        f_vektor[a] = f_vektor_1[i]
        for j in range(anzahl_dof):
            if j not in dof_gehalten:
                m_matrix[a][b] = m_matrix_1[i][j]
                k_matrix[a][b] = k_matrix_1[i][j]
                b += 1
        a += 1
```

Abbildung 6.18: Code-Ausschnitt, Implementation der Steifigkeit der Einspannung

Die Iteration wird abgebrochen, sobald sich die berechnete und die gemessene erste Eigenfrequenz um weniger als 0.001 Hz unterscheiden. Um zu verhindern, dass eine Endlosschleife entstehen könnte, wird die Anzahl Iterationen auf 50 beschränkt (siehe „q“ in Abbildung 6.17). Die Zahl 50 ergab sich aus der Erfahrung mit den bisher untersuchten Masten. Die Vorgehensweise, wie der Wert der Steifigkeit verändert wird, ist relativ simpel: Falls die berechnete Frequenz zu klein ist, wird der Wert der Steifigkeit erhöht, falls die Frequenz dagegen zu gross ist, wird die Steifigkeit verringert. Im ersten Durchgang wird ein geringer Wert von 10 kNm/rad für die Steifigkeit angenommen. Die „veraenderung“ (siehe Abbildung 6.19) der Steifigkeit erfolgt zuerst in Schritten von 1'000 kNm/rad. Jedes Mal, wenn die Richtung der Veränderung nicht mit der vorhergegangenen Iteration übereinstimmt (neu Subtraktion anstatt Addition oder umgekehrt), wird der Betrag der Veränderung halbiert bis schlussendlich ein Wert für die Steifigkeit gefunden wird.

```

if eigenfrequenzen[0] < f_1_gemessen + 0.001 and eigenfrequenzen[0] > f_1_gemessen - 0.001:
    print("geschafft")
    break
if eigenfrequenzen[0] < f_1_gemessen:
    if groesser == False:
        veraenderung = veraenderung/2
        k_bodenfeder += veraenderung
        groesser = True
if eigenfrequenzen[0] > f_1_gemessen:
    if groesser == True:
        veraenderung = veraenderung/2
        k_bodenfeder -= veraenderung
        groesser = False

```

Abbildung 6.19: Code-Ausschnitt, Anpassung der Steifigkeit

6.6 Statische Deformation und Eigenfrequenzen mit tatsächlicher Einspannung

```

#statische Deformation, Eigenfrequenzen und -modi mit der berechneten Steifigkeit berechnen
#jetzt wurde k_bodenfeder bestimmt: Die gesamte berechnung wird noch einmal mit k_bodenfeder durchgeführt und ausgegeben
print("Resultate unter Berücksichtigung der Einspannung im Boden:")
print("Federsteifigkeit der Einspannung [kNm/rad]:", k_bodenfeder/1000)
sheet.append([])
sheet.append(["Resultate unter Berücksichtigung der Einspannung im Boden:"])
sheet.append(["Federsteifigkeit der Einspannung [kNm/rad]:", k_bodenfeder/1000])
#die folgende Berechnung wird 2 mal durchgeführt: zuerst um die x-Achse, danach um die y-Achse
for achse in range(2):
    #I, Flächenträgheitsmoment um die x-Achse oder um die y-Achse, je nach Berechnungsrichtung
    I = [0] * (anzahl_elemente)
    for i in range(anzahl_elemente):
        if achse == 0:
            I[i] = I_x[i]
        if achse == 1:
            I[i] = I_y[i]
    anzahl_nodes = anzahl_elemente + 1
    anzahl_dof = 3*anzahl_nodes
    k_matrix_1 = np.zeros( (anzahl_dof,anzahl_dof) )
    m_matrix_1 = np.zeros( (anzahl_dof,anzahl_dof) )
    for i in range(anzahl_elemente):
        k_1a = np.array( [ [E*A[i]/L, 0, 0],
                           [0, 12*E*I[i]/(L**3), 6*E*I[i]/(L**2)],
                           [0, 6*E*I[i]/(L**2), 4*E*I[i]/L ] ] )
        k_1b = np.array( [ [-E*A[i]/L, 0, 0],
                           [0, -12*E*I[i]/(L**3), 6*E*I[i]/(L**2)],
                           [0, 6*E*I[i]/(L**2), 2*E*I[i]/L ] ] )
        k_1c = np.array( [ [-E*A[i]/L, 0, 0],
                           [0, -12*E*I[i]/(L**3), -6*E*I[i]/(L**2)],
                           [0, 6*E*I[i]/(L**2), 2*E*I[i]/L ] ] )
        k_1d = np.array( [ [E*A[i]/L, 0, 0],
                           [0, 12*E*I[i]/(L**3), -6*E*I[i]/(L**2)],
                           [0, -6*E*I[i]/(L**2), 4*E*I[i]/L ] ] )

```

Abbildung 6.20: Code-Ausschnitt, Mastmodellierung mit tatsächlicher Steifigkeit

Der letzte Teil der Berechnung beginnt mit dem Eintrag der berechneten Steifigkeit in das workbook „sheet“ (siehe Abbildung 6.20), damit dem Benutzer dieses Resultat angezeigt wird. Der Code, welcher dann folgt, entspricht grösstenteils exakt dem, welcher im Unterkapitel 6.4 aufgeführt ist. Der Mast wird wieder zweimal (zur Berücksichtigung der x- und y-Richtung) modelliert und die statische Deformation wird ebenso berechnet wie die Eigenfrequenzen und -modi. Nur der Rotationsfreiheitsgrad des ersten Knotens unterscheidet sich. Dieser wird nicht mehr als blockiert betrachtet. Stattdessen wird, wie im Unterkapitel 6.5 beschrieben, die Steifigkeit der Einspannung in die Steifigkeitsmatrix ergänzt. Erneut werden pro Richtung die statischen Deformationen sowie die ersten fünf Eigenfrequenzen und -modi zu „sheet“ hinzugefügt.

6.7 Programmabschluss

```
#Titel für plots schreiben
for i in range (13):
    sheet.append([])
sheet.append(["statische Deformationen in y- und x-Richtung"])
sheet.append(["ohne Berücksichtigung der Einspannung:"])
for i in range (24):
    sheet.append([])
sheet.append(["Eigenmodi 1-5 in x-Richtung"])
sheet.append(["ohne Berücksichtigung der Einspannung:"])
for i in range (24):
    sheet.append([])
sheet.append(["Eigenmodi 1-5 in y-Richtung"])
sheet.append(["ohne Berücksichtigung der Einspannung:"])

if erfüllt == False:
    for i in range (24):
        sheet.append([])
    sheet.append(["statische Deformationen in y- und x-Richtung"])
    sheet.append(["mit Berücksichtigung der Einspannung:"])
    for i in range (24):
        sheet.append([])
    sheet.append(["Eigenmodi 1-5 in x-Richtung"])
    sheet.append(["mit Berücksichtigung der Einspannung:"])
    for i in range (24):
        sheet.append([])
    sheet.append(["Eigenmodi 1-5 in y-Richtung"])
    sheet.append(["mit Berücksichtigung der Einspannung:"])

#Inputkontrolle 2
elif inputkontrolle == False:
    print("Input ist falsch.")
    sheet.append(["Bitte überprüfen sie den Input."])

#workbook speichern
workbook.save('Resultate_Frequenzanalyse.xlsx') #Die neue Excel Datei wird im gleichen Ordner gespeichert wie die Jupyter Datei
```

Abbildung 6.21: Code-Ausschnitt, Programmabschluss

Das Programm endet mit drei kurzen Abschnitten (siehe Abbildung 6.21): Zuerst werden, in Abhängigkeit der Anzahl der erstellten Plots, die Beschreibungen der Plots an den entsprechenden Stellen eingefügt. Im Abschnitt „Inputkontrolle 2“ wird eine Fehlermeldung generiert, falls bei den Benutzereingaben ein möglicher Fehler erkannt wurde. Unter „workbook speichern“ wird das bisher verwendete workbook „sheet“ als Excel-Datei mit dem Namen „Resultate_Standicherheit“ im gleichen Ordner wie die das Python-Programm gespeichert. Falls der Benutzer einen anderen Speicherort bevorzugt, kann der Dateipfad an dieser Stelle eingefügt werden.

Literaturverzeichnis

- [1] CEN, “Lichtmaste - Teil 3-3: Bemessung und Nachweis - Rechnerischer Nachweis,” 2013, SN EN 40-3-3.
- [2] Schweizerischer Ingenieur- und Architektenverein, “Einwirkungen auf Tragwerke,” 2020, SIA 261.
- [3] CEN, “Lichtmaste - Teil 3-1: Bemessung und Nachweis - Charakteristische Werte der Lasten,” 2013, SN EN 40-3-1.
- [4] S. Piccolo, D. Sadiku, “Berechnung der Windlasten von Beleuchtungsmasten,” 2023, Institut für Baustatik und Konstruktion, ETH Zürich.
- [5] CEN, “Eurocode 1 - Einwirkungen auf Tragwerke - Teil 1-4: Allgemein Einwirkungen - Windlasten - Nationaler Anhang NA zur SN eN 1991-1-4,” 2016, SN EN 1991-1-4/NA.
- [6] ETH Zürich, Institut für Baustatik und Konstruktion, “Method of Finite Elements I,” 3D Euler-Bernoulli beam.
- [7] E. Chatzi, “Baustatik II - Kapitel Vb, die direkte Steifigkeitsmethode, Hauptschritte Anwendung bei Fachwerksystemen,” 2023, Institute of Structural Engineering, ETH Zürich.
- [8] D. Kochmann, “Dynamics (Fall Semester),” 2022, Mechanics Material Lab, Institute of Mechanical Systems, ETH Zürich.
- [9] S. Johnson, “Physics HS2022, Lecture notes, v1.01,” 2022, ETH Zürich.

A Anhang

Dieser Link ermöglicht den Zugriff auf eine Polybox, in welcher sich unter anderem die Input-Datei und das Python-Programm befinden:

<https://polybox.ethz.ch/index.php/s/xexkqD6ZMQy3NZT>